

# A reinforced hybrid genetic algorithm for the traveling salesman problem

Jiongzhi Zheng, Jialun Zhong, Menglei Chen, Kun He \*

School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China

## ARTICLE INFO

### Keywords:

Combinatorial optimization  
Traveling salesman problem  
Hybrid genetic algorithm  
Local search  
Reinforcement learning

## ABSTRACT

We propose a new method called the Reinforced Hybrid Genetic Algorithm (RHGA) for solving the famous NP-hard Traveling Salesman Problem (TSP). Specifically, we combine reinforcement learning with the well-known Edge Assembly Crossover Genetic Algorithm (EAX-GA) and the Lin–Kernighan–Helsgaun (LKH) local search heuristic. In the hybrid algorithm, LKH can help EAX-GA improve the population by its effective local search, and EAX-GA can help LKH escape from local optima by providing high-quality and diverse initial solutions. We restrict that there is only one special individual among the population in EAX-GA that can be improved by LKH. Such a mechanism can prevent the population diversity, efficiency, and algorithm performance from declining due to the redundant calling of LKH upon the population. As a result, our proposed hybrid mechanism can help EAX-GA and LKH boost each other's performance without reducing the convergence rate of the population. The reinforcement learning technique based on the Q-learning algorithm further promotes the hybrid genetic algorithm. Experimental results on 138 well-known and widely used TSP benchmarks with the number of cities ranging from 1,000 to 85,900 demonstrate the excellent performance of the proposed RHGA algorithm.

## 1. Introduction

Given a complete, undirected graph  $G = (V, E)$ , where  $V = \{1, 2, \dots, n\}$  denotes the set of  $n$  cities and  $E = \{(i, j) | i, j \in V\}$  denotes the set of all pairwise edges,  $d(i, j)$  represents the distance (cost) of edge  $(i, j)$ , i.e., the distance of traveling from city  $i$  to city  $j$ , the Traveling Salesman Problem (TSP) aims to find a Hamiltonian cycle represented by a permutation  $(s_1, s_2, \dots, s_n)$  of cities  $\{1, 2, \dots, n\}$  that minimizes the total distance:  $d(s_1, s_2) + d(s_2, s_3) + \dots + d(s_{n-1}, s_n) + d(s_n, s_1)$ . The TSP is one of the most famous and well-studied NP-hard combinatorial optimization problems, which is very easy to understand but very difficult to solve to optimality. Over the years, the TSP has become a touchstone in the field of combinatorial optimization.

Typical methods for solving the TSP can be categorized into exact algorithms, approximation algorithms, and heuristics. Exact algorithms may be prohibitive for large instances, and approximation algorithms may suffer from weak optimal guarantees or empirical performance (Khalil et al., 2017). Heuristics are known to be the most efficient and effective approaches for solving the TSP. Two of the state-of-the-art heuristics are the Lin–Kernighan–Helsgaun (LKH) local search algorithm (Helsgaun, 2000) and the Edge Assembly Crossover Genetic Algorithm (EAX-GA) (Nagata and Kobayashi, 2013). Both of them provide the best-known solutions on many TSP benchmark instances. Their variants or extended algorithms are also used to solve various

variants of the TSP (Helsgaun, 2017; Nagata et al., 2010; He and Hao, 2022).

As two representative heuristic algorithms, both LKH and EAX-GA have advantages and disadvantages. For example, EAX-GA is very efficient and powerful in solving TSP instances with tens to hundreds of thousands of cities, providing the best-known solutions of the six famous instances with 100,000 to 200,000 cities in the Art TSP benchmarks.<sup>1</sup> But EAX-GA is hard to scale to super large instances, such as TSP instances with millions of cities, since the convergence of the population is too time-consuming. As an efficient local search algorithm, LKH can yield near-optimal solutions faster than EAX-GA does. It is also suitable for TSP instances with various scales, especially for super large instances, providing the best-known solution of the famous World TSP instance with 1,904,711 cities.<sup>2</sup> However, LKH is not as good as EAX-GA in solving the TSPs with 10,000 to 200,000 cities, since the population can help EAX-GA explore the solution space better than LKH does for TSP instances with such large scales. Based on these characteristics, a straightforward idea is to find a reasonable way to combine EAX-GA with LKH to exploit their complementary, so as to help them boost each other.

There have been related studies trying to combine EAX-GA with LKH or its predecessor, the LK heuristic (Lin and Kernighan, 1973). For example, Tsai et al. (2004) propose to combine the earliest version

\* Corresponding author.

E-mail address: [brooklet60@hust.edu.cn](mailto:brooklet60@hust.edu.cn) (K. He).

<sup>1</sup> <http://www.math.uwaterloo.ca/tsp/data/art/index.html>.

<sup>2</sup> <http://www.math.uwaterloo.ca/tsp/world/index.html>.

of EAX-GA (Nagata and Kobayashi, 1997) with LK. Their proposed algorithm HeSEA reports better results than EAX-GA, LK, and LKH in solving TSP instances with up to 15,112 cities. However, HeSEA follows the similar hybrid mechanism of many other hybrid algorithms (Ulder et al., 1990; Freisleben and Merz, 1996; Merz and Freisleben, 1997; Nguyen et al., 2007; Whitley et al., 2010; Wang, 2014; Rashid and Mosteiro, 2017; Ilin et al., 2020) that combine genetic algorithms with LK-based algorithms (or other local search methods like 2-opt). That is, applying the local search methods to optimize every individual in the current population or every surviving offspring generated. Such a mechanism has two disadvantages: (1) the population diversity will be broken because the local optimal solutions (of different tours) calculated by the same local search method are similar. (2) It is very time-consuming to frequently apply local search methods to calculate the local optimal solutions, as HeSEA (Tsai et al., 2004) shows worse efficiency and reports longer computation time than LK and LKH for large scale instances.

In addition, Kerschke et al. (2018) propose to combine several TSP solvers, including EAX-GA and LKH, by a machine learning model based on supervised learning. The machine learning model can help their proposed hybrid solver select an appropriate solver to solve the input TSP instance. Such a hybrid is a simple combination of the TSP solvers, in which the solvers do not interact with each other. The solution of the hybrid solver (consisting of only EAX-GA and LKH) is bounded by the better one of the solutions obtained by EAX-GA and LKH.

In this paper, we propose a reinforcement learning (Sutton and Barto, 1998; Mazyavkina et al., 2021; Gama and Fernandes, 2021) based hybrid genetic algorithm for the TSP, called Reinforced Hybrid Genetic Algorithm (RHGA), that combines EAX-GA with the LKH local search and further applies reinforcement learning to improve the performance. In the proposed RHGA, there is only one special individual (e.g., the first individual) in the population of EAX-GA that can be improved by the local search algorithm of LKH, because the redundant local search operations and redundant local optimal solutions of LKH in the population may reduce the population diversity and the efficiency, as well as the solution quality of the genetic algorithm. Moreover, our proposed combination mechanism can make full use of the complementary of EAX-GA and LKH, and help them boost each other. As a result, the hybrid mechanism in our proposed algorithm can fix the aforementioned issues of the hybrid mechanisms in the existing hybrid genetic algorithms (Ulder et al., 1990; Freisleben and Merz, 1996; Merz and Freisleben, 1997; Tsai et al., 2004; Nguyen et al., 2007; Whitley et al., 2010; Wang, 2014; Rashid and Mosteiro, 2017; Ilin et al., 2020) and hybrid solver (Kerschke et al., 2018) for the TSP and fits well with EAX-GA and LKH.

Moreover, we apply reinforcement learning to further improve the performance of the proposed hybrid genetic algorithm. We apply the technique proposed by Zheng et al. (2021) that employs reinforcement learning to learn an adaptive Q-value during the local search process of LKH as a metric for evaluating the quality of the edges. We use the adaptive Q-value learned by the Q-learning algorithm (Sutton and Barto, 1998) to replace the important evaluation metrics of the edges used in the key steps in both LKH and EAX-GA. In this way, both LKH and EAX-GA can be enhanced by the reinforcement learning in our RHGA algorithm. Related studies of (reinforcement) learning based methods for the TSP are referred to Section 2.1, where we also describe the advantages of our reinforcement learning method over them.

The main contributions of this work are as follows:

- We propose a creative and distinctive hybrid mechanism to combine two of the state-of-the-art TSP heuristic algorithms, EAX-GA and LKH, through a special individual. In the proposed RHGA algorithm, EAX-GA and LKH can boost each other with the help of the special individual.
- We propose to combine reinforcement learning with the key steps of both EAX-GA and LKH to further improve the performance of the hybrid genetic algorithm. The adaptive Q-value learned by the Q-learning algorithm significantly outperforms the metrics used in EAX-GA and LKH for evaluating the quality of the edges.
- Experimental results on 138 well-known and widely used TSP benchmarks with the number of cities ranging from 1000 to 85,900 demonstrate the promising performance of our proposed algorithm.

## 2. Related works

For related works, we first introduce (reinforcement) learning based algorithms for solving the TSP, then briefly introduce the two state-of-the-art heuristic algorithms for solving the TSP, EAX-GA and LKH, which will also be incorporated into our proposed algorithm. For details of these two algorithms, we refer to Nagata and Kobayashi (2013) and Helsgaun (2000).

### 2.1. Learning based algorithms for the TSP

(Reinforcement) learning based methods for the TSP can be divided into two categories. The first category is end-to-end methods (Bengio et al., 2021), which are usually based on deep neural networks. When receiving an input TSP instance, they use the trained learning model to generate a solution directly. For example, Bello et al. (2017) address TSP by using the actor-critic method to train a pointer network (Vinyals et al., 2015). The S2V-DQN algorithm (Khalil et al., 2017) applies reinforcement learning to train a graph neural network so as to solve several combinatorial optimization problems, including minimum vertex cover, maximum cut, and TSP. Xin et al. (2021a) propose to train multiple diverse policies to construct the TSP tour, which effectively increases the chance of finding good solutions compared with the methods that train only one policy. These methods provide good innovations in the field of applying machine learning to solve combinatorial optimization problems. As for the performance, they can yield near-optimal or optimal solutions for the TSP instances with less than hundreds of cities. However, they are usually hard to scale to large instances (with more than thousands of cities) due to the complexity of the deep neural networks.

Methods belonging to the second category combine (reinforcement) learning methods with traditional algorithms. Some of them use traditional algorithms as the core and frequently call the learning models to help explore the solution space or guide the search direction. For example, Liu and Zeng (2009) employ reinforcement learning to construct mutation individuals in the previous version of EAX-GA (Nagata, 2006) and report better results than EAX-GA and LKH on instances with up to 2392 cities. But the efficiency of their proposed algorithm is not as good as that of LKH. de O da Costa et al. (2020) and Sui et al. (2021) use deep reinforcement learning to guide 2-opt and 3-opt local search operators, and report results on instances with no more than 500 cities. Other methods separate the learning models and traditional algorithms. They first apply (reinforcement) learning methods to yield initial solutions (Zhao et al., 2021) or some configuration information (Xin et al., 2021b), and then use traditional algorithms to find high-quality solutions following the obtained initial solutions or information. Among them, the NeuroLKH algorithm (Xin et al., 2021b) is one of the state-of-the-art, which uses a Sparse Graph Network with supervised learning to generate the candidate edges for LKH. It reports better or similar results compared with LKH in instances with less than 6000 cities.

In summary, (reinforcement) learning based methods with deep neural networks for the TSP may suffer from the bottleneck of hardly solving large scale instances, and the combination of traditional reinforcement learning methods (training tables, not deep neural networks) with existing (heuristic) algorithms may reduce the efficiency of the

**Algorithm 1:** EAX-GA( $N_{pop}, N_{ch}$ )

---

**Input:** population size:  $N_{pop}$ , number of offsprings produced by a pair of parents:  $N_{ch}$   
**Output:** output solution:  $x_{out}$

```

1  $\{x_1, x_2, \dots, x_{N_{pop}}\} \leftarrow \text{Generate\_Initial\_Pop}();$ 
2 while a termination condition is not satisfied do
3    $rp(\cdot) \leftarrow$  a random permutation of  $\{1, 2, \dots, N_{pop}\};$ 
4   for  $i \leftarrow 1 : N_{pop}$  do
5      $p_A \leftarrow x_{rp(i)}, p_B \leftarrow x_{rp(i+1)};$ 
6      $\{ch_1, ch_2, \dots, ch_{N_{ch}}\} \leftarrow \text{EAX}(p_A, p_B);$ 
7      $x_{rp(i)} \leftarrow \text{Select\_Best}(ch_1, \dots, ch_{N_{ch}}, p_A);$ 
8  $x_{out} \leftarrow$  the shortest tour in the population;
9 return  $x_{out};$ 

```

---

algorithm. The reinforcement learning method in our proposed RHGA algorithm can avoid these issues. On the one hand, instead of using deep neural networks, our reinforcement learning method uses the traditional Q-learning algorithm (Sutton and Barto, 1998) to train a table. Therefore, our algorithm can solve very large instances, as we tested RHGA on instances with up to 85,900 cities. On the other hand, we combine reinforcement learning with the core search steps of LKH and EAX-GA in a reasonable way, which prevents the reduction of the algorithm's efficiency. The experimental results show that RHGA significantly outperforms the newest versions of LKH and EAX-GA within similar calculation time.

## 2.2. Edge assembly crossover genetic algorithm

The EAX-GA algorithm (Nagata and Kobayashi, 2013) generates offspring solutions by combining edges from the two parent solutions and adding relatively few new short edges determined by a simple search procedure that is similar to the 2-opt local search. The core of EAX-GA is its edge assembly crossover (EAX) operation. The main flow of EAX-GA is presented in Algorithm 1. EAX-GA first generates the initial population that contains  $N_{pop}$  individuals by the function *Generate\_Initial\_Pop()*, which is a greedy 2-opt local search method (Nagata and Kobayashi, 2013), and then repeats the process of crossover and offspring selection until a termination condition (see below for details) is satisfied.

At each generation of EAX-GA, each individual in the population is selected once as parent  $p_A$  and once as parent  $p_B$ , in a random order (lines 3–5). The crossover operation EAX is applied to parents  $p_A$  and  $p_B$  to produce  $N_{ch}$  offsprings (line 6). Finally, the best individual among the  $N_{ch}$  offsprings coupled with parent  $p_A$  is selected to replace  $p_A$  according to a given evaluation function (line 7) based on the edge entropy measure (Maekawa et al., 1996). Applying the edge entropy measure rather than the straightforward tour length measure can significantly improve the diversity of the population.

Let  $E_A \subset E$  and  $E_B \subset E$  be the sets of edges corresponding to  $p_A$  and  $p_B$ , the EAX operation generates offsprings through the following six steps, as illustrated in Fig. 1.

- **Step 1:** Construct an undirected multigraph  $G_{AB} = (V, E_A \cup E_B)$  by combining all the edges of  $E_A$  and  $E_B$ . The edges belonging to either  $E_A$  or  $E_B$  in  $G_{AB}$  are labeled.
- **Step 2:** Randomly partition all edges of  $G_{AB}$  into AB-cycles, where an AB-cycle consists of alternately linked edges of  $E_A$  and  $E_B$ .
- **Step 3:** Construct an E-set by selecting AB-cycles according to a given selection strategy, where an E-set is defined as the union of AB-cycles.
- **Step 4:** Generate an intermediate solution from  $p_A$  by removing the edges of  $E_A$  and adding the edges of  $E_B$  in the E-set. Let  $E_C = (E_A \setminus (E\text{-set} \cap E_A)) \cup (E\text{-set} \cap E_B)$  be the set of edges in the

intermediate solution. An intermediate solution consists of one or more sub-tours and may not be a feasible solution for TSP.

- **Step 5:** Connect all sub-tours into a tour to generate a valid offspring. This step merges the smallest sub-tour (the sub-tour with the least number of edges) with other sub-tours each time. Let  $U$  be the set of edges in the smallest sub-tour, the goal is to find 4-tuples of edges  $\{e^*, e'^*, e''^*, e'''^*\} = \arg \min_{e \in U, e' \in E_C \setminus U} \{-d(e) - d(e') + d(e'') + d(e''')\}$ , where  $e$  and  $e'$  denote two edges to be removed, and  $e''$  and  $e'''$  denote two edges to be added to connect the breakpoints. Then the sub-tours are connected by  $E_C \leftarrow (E_C \setminus \{e^*, e'^*\}) \cup \{e''^*, e'''^*\}$ . In particular, EAX-GA restricts the search to promising pairs of  $e$  and  $e'$  to reduce the search scope and improve the algorithm's efficiency. For each  $e \in U$ , the candidates of  $e'$  are restricted to a set of edges that satisfy the following condition: at least one end of  $e'$  is among the  $N_{near}$  (10 by default) closest to either end of  $e$ .
- **Step 6:** Loop steps 3–5 until  $N_{ch}$  offsprings are generated. Then terminate the procedure.

Note that the metric for determining the candidates of  $e'$  in Step 5 is the distance. This metric is very important since it determines the new edges that can be added to the population. In the proposed RHGA algorithm, we replace the distance metric used here with the Q-value learned by the Q-learning algorithm to improve the performance.

The EAX-GA algorithm consists of two stages. It terminates stage I when no improvement in the best solution is found over a period of generations, and then switches to stage II. Specifically, let  $Gen$  be the number of generations at which no improvement in the best solution is found over the recent  $1500/N_{ch}$  generations. If the value of  $Gen$  has already been determined and the best solution does not improve over the last  $G_{max} = Gen/10$  generations, EAX-GA terminates stage I and proceeds to stage II. Stage II is also terminated by the same condition (both  $Gen$  and  $G_{max}$  should be recalculated in this stage).

The only difference between the two stages is the selection strategy of the E-set (Step 3) during the EAX crossover process. In stage I, a single AB-cycle is selected randomly as the E-set without overlapping with the previous selections. Such a strategy is very simple, and thus computationally efficient. In stage II, the *block2* strategy (Nagata and Kobayashi, 2013) is applied, which is effective in solving large TSP instances. Its basic idea is to construct an E-set by selecting AB-cycles so that the resulting intermediate solution consists of relatively few sub-tours and the resulting offspring consists of more edges of  $p_B$ . The intermediate solution with few sub-tours corresponds to an offspring that inherits its parents well, and making the offspring inherit more edges of  $p_B$  can prevent the algorithm from falling into the local optima easily.

## 2.3. Lin-Kernighan-Helsgaun algorithm

LKH uses the  $k$ -opt heuristic (Lin, 1965) as the optimization method to find high-quality solutions. The  $k$ -opt in LKH replaces at most  $k_{max}$  (5 by default) edges in the current tour with the same number of new edges, and restricts that the edges to be added must be selected from the candidate sets, so as to reduce the search scope and improve the efficiency. This subsection first introduces two important parts of LKH, i.e., the method of creating the candidate sets and the  $k$ -opt process, then introduces the main process of LKH.

### 2.3.1. Candidate sets in LKH

In LKH, each city has its candidate set that records several candidate cities. Let  $CS^i$  be the candidate set of city  $i$  ( $i \in V$ ), LKH restricts that the edges to be added in the  $k$ -opt process must be selected from the set  $\{(i, j) \in E | j \in CS^i \vee i \in CS^j\}$ . LKH proposes an  $\alpha$ -value to evaluate the quality of the edges, and applies the  $\alpha$ -value as the metric for selecting and sorting candidate cities. The  $\alpha$ -value is defined from the structure of 1-tree (Held and Karp, 1970). A 1-tree for the graph  $G = (V, E)$  is a

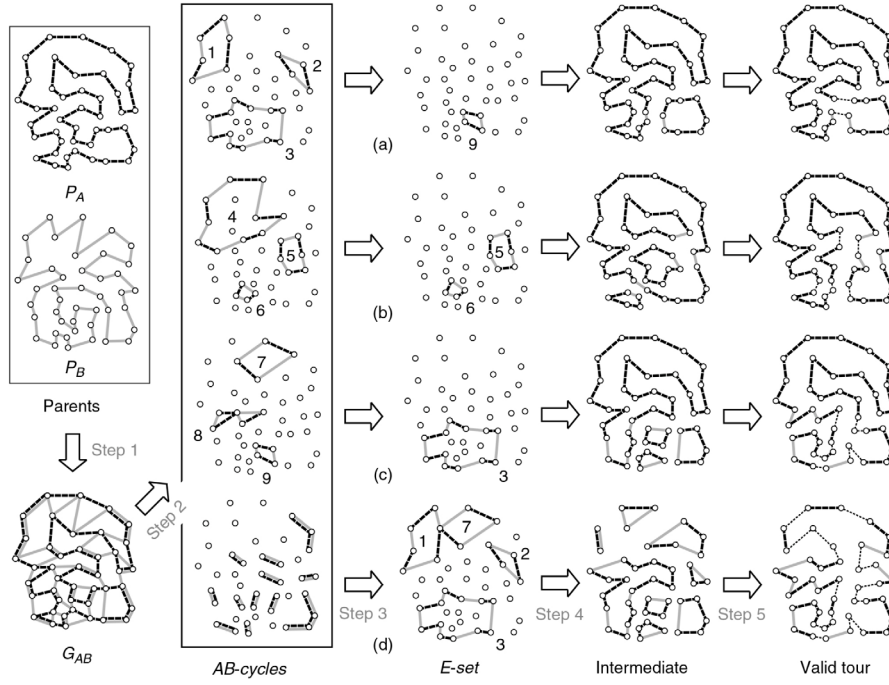
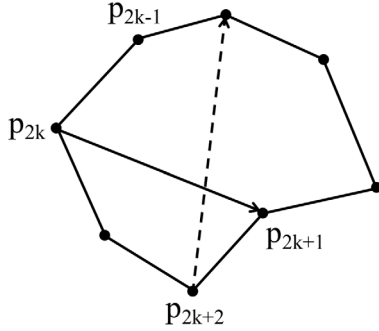


Fig. 1. Illustration of the EAX operation (Nagata and Kobayashi, 2013)..

Fig. 2. The  $k$ -opt process of LKH.

spanning tree on the node set  $V \setminus \{v\}$  combined with two edges from  $E$  incident to a node  $v$  chosen arbitrarily. The minimum 1-tree is the 1-tree with the minimum length (the length of a 1-tree is the total length of its contained edges). Obviously, the length of the minimum 1-tree is a lower bound of the optimal TSP solution. The equation for calculating the  $\alpha$ -value of an edge  $(i, j)$  is as follows:

$$\alpha(i, j) = L(T^+(i, j)) - L(T), \quad (1)$$

where  $L(T)$  is the length of the minimum 1-tree of the graph  $G$ , and  $L(T^+(i, j))$  is the length of the minimum 1-tree required to contain edge  $(i, j)$ . The candidate set of each city in LKH records five (default value) other cities with the smallest  $\alpha$ -value to this city in ascending order. The advantage of the candidate set is further enhanced by adding penalties to the cities. Details about the penalties are referred to Helsgaun (2000).

### 2.3.2. The $k$ -opt in LKH

Fig. 2 illustrates the process of the  $k$ -opt heuristic in LKH. The  $k$ -opt process is actually a partial depth-first search process, that the maximum depth of the search tree is restricted to  $k_{max}$  (5 by default).

#### Algorithm 2: $k$ -opt( $x_{in}, p_1, p, k$ )

---

**Input:** input solution:  $x_{in}$ , starting city:  $p_1$ , sequence of the corresponding cities:  $p$ , current search depth:  $k$

**Output:** output solution  $x_{out}$ , sequence of the corresponding cities  $p$

---

```

1 for each  $p_{2k}$  linked with  $p_{2k-1}$  in  $x_{in}$  do
2   if  $p_{2k}$  does not satisfy the constraint C-I then
3     continue;
4   if  $k \geq 2 \wedge \sum_{j=1}^k d(p_{2j-1}, p_{2j}) < \sum_{j=1}^{k-1} d(p_{2j}, p_{2j+1}) + d(p_{2k}, p_1)$ 
5     then
6        $x_{out} \leftarrow x_{in}$ ;
7       for  $j \leftarrow 1 : k$  do
8         remove edge  $(p_{2j-1}, p_{2j})$  from  $x_{out}$ ;
9       for  $j \leftarrow 1 : k - 1$  do
10        add edge  $(p_{2j}, p_{2j+1})$  into  $x_{out}$ ;
11      Add edge  $(p_{2k}, p_1)$  into  $x_{out}$ ;
12      return  $(x_{out}, p)$ ;
13   if  $k = k_{max}$  then return  $(x_{in}, \emptyset)$ ;
14   for  $j \leftarrow 1 : 5$  do
15      $p_{2k+1} \leftarrow$  the  $j$ -th city in  $CS^{p_{2k}}$ ;
16     if  $p_{2k+1}$  does not satisfy constraint C-II then
17       continue;
18      $(x_{temp}, p') \leftarrow k\text{-opt}(x_{in}, p_1, p \cup \{p_{2k}, p_{2k+1}\}, k + 1)$ ;
19     if  $l(x_{temp}) < l(x_{in})$  then
20       return  $(x_{temp}, p')$ ;
21 return  $(x_{in}, \emptyset)$ ;

```

---

The  $k$ -opt process starts from a starting city  $p_1$  (i.e., root of the search tree), then alternatively selects an edge to be removed, i.e., edge  $(p_{2k-1}, p_{2k})$ , and an edge to be added, i.e., edge  $(p_{2k}, p_{2k+1})$ , until the maximum search depth is reached or a  $k$ -opt move that can improve the current tour is found. Note that these edges are connected, thus



**Algorithm 3: LKH( $M$ )**


---

**Input:** the maximum number of iterations:  $M$   
**Output:** output solution:  $x_{out}$

```

1 Create_Candidate_Set();
2 Initialize the length of the best solution  $l(x_{best}) \leftarrow +\infty$ ;
3 for  $i \leftarrow 1 : M$  do
4    $x_{init} \leftarrow \text{Choose\_Initial\_Tour}()$ ,  $x_{temp} \leftarrow x_{init}$ ;
5   Initialize the set of the cities that have not been selected as
   the starting city of the  $k$ -opt:  $A \leftarrow \{1, 2, \dots, n\}$ ;
6   while TRUE do
7     if  $A = \emptyset$  then break;
8      $p_1 \leftarrow$  a random city in  $A$ ,  $A \leftarrow A \setminus \{p_1\}$ ;
9      $(x'_{temp}, p) \leftarrow k\text{-opt}(x_{temp}, p_1, \{p_1\}, 1)$ ;
10    if  $l(x'_{temp}) < l(x_{temp})$  then
11       $x_{temp} \leftarrow x'_{temp}$ ;
12      for  $j \leftarrow 1 : |p|$  do  $A \leftarrow A \cup \{p_j\}$ ;
13    if  $l(x_{temp}) < l(x_{best})$  then  $x_{best} \leftarrow x_{temp}$ ;
14 return  $x_{best}$ ;
```

---

selecting the involved edges in  $k$ -opt can be regarded as selecting a sequence (cycle) of cities. The selection of the cities  $p_{2k}$  and  $p_{2k+1}$  should satisfy the following constraints:

- **C-I:** for  $k \geq 2$ , connecting  $p_{2k}$  back to  $p_1$  should result in a feasible TSP tour.
- **C-II:**  $p_{2k+1}$  is always chosen so that  $\sum_{j=1}^i (d(p_{2k-1}, p_{2k}) - d(p_{2k}, p_{2k+1})) > 0$ .

Let  $l(x)$  be the length of solution  $x$ , the procedure of the  $k$ -opt process is presented in Algorithm 2. As shown in Algorithm 2, the  $k$ -opt process tries to improve the current solution by traversing the partial depth-first search tree from the root  $p_1$ . When selecting the edge to be removed, i.e., edge  $(p_{2k-1}, p_{2k})$  (the same as selecting  $p_{2k}$  from  $p_{2k-1}$ ), the algorithm traverses the two cities connected with city  $p_{2k-1}$  in the current TSP tour (line 1). When selecting the edge to be added, i.e., edge  $(p_{2k}, p_{2k+1})$  (the same as selecting  $p_{2k+1}$  from  $p_{2k}$ ), the algorithm traverses the candidate set of city  $p_{2k}$  (lines 13–14), and the constraint C-II is applied as a smart pruning strategy to improve the efficiency (lines 15–16). Once a  $k$ -opt move that can improve the current solution is found, the algorithm performs this move on  $x_{in}$  and outputs the resulting solution  $x_{out}$  (lines 4–11).

### 2.3.3. Main process of LKH

Finally, we introduce the main process of the LKH algorithm, which is presented in Algorithm 3. The *Create\_Candidate\_Set()* function (line 1) calculates the  $\pi$ -values and  $\alpha$ -values, and creates the candidate sets according to the method described in Section 2.3.1. At each iteration (lines 3–13), an initial solution  $x_{init}$  is generated, and the LKH algorithm uses the  $k$ -opt heuristic (Algorithm 2) to improve the current solution  $x_{temp}$  until the local optimum is reached (lines 6–12), i.e.,  $x_{temp}$  cannot be improved by the  $k$ -opt heuristic starting from any starting city  $p_1 \in \{1, 2, \dots, n\}$ .

## 3. The proposed algorithm

In the proposed Reinforced Hybrid Genetic Algorithm (RHGA), we design a novel hybrid mechanism with a special individual as the core to combine EAX-GA with the LKH local search. The EAX-GA and LKH can boost each other with the help of the special individual. The reinforcement learning technique (Zheng et al., 2021) is combined with the key steps of both LKH and EAX-GA to further improve the hybrid genetic algorithm, by replacing the evaluation metrics for the edges

used in LKH ( $\alpha$ -value) and EAX-GA (distance) with the learned adaptive Q-value.

This section first introduces how the Q-value (i.e., reinforcement learning) is used in RHGA, then introduces the reinforced LKH local search method (Q-LKH) in RHGA, and describes the main process of RHGA that contains the description of the proposed hybrid mechanism, finally concludes the advantages of RHGA.

### 3.1. The Q-value in RHGA

The Q-value in RHGA actually determines the candidate edges in both LKH and EAX-GA. Note that the larger the Q-value of an edge, the higher the quality of the edge. The candidate set of each city in RHGA records  $K$  (25 by default) of other cities with the largest Q-values to this city in descending order. When selecting an edge to be added ( $p_{2k}, p_{2k+1}$ ) during the  $k$ -opt process in the Q-LKH local search component of RHGA,  $p_{2k+1}$  can only be selected among the top five (default value as in LKH) cities in the candidate set of  $p_{2k}$ . Similarly, when merging two sub-tours during the offspring generating process in the EAX-GA component of RHGA, the two edges to be removed,  $e$  and  $e'$ , must satisfy that at least one end of  $e'$  is among the top ten (default value as in EAX-GA) cities in the candidate set of either end of  $e$ .

RHGA designs an initial Q-value for each edge to generate the initial candidate sets. Before calculating the initial Q-value, the algorithm needs to calculate the lower bound of the optimal TSP solution  $L(T)$  (see Eq. (1)) and the  $\alpha$ -values corresponding to  $L(T)$ , by the method in LKH (see Section 2.3.1). Then the initial Q-value for edge  $(i, j)$  can be calculated by:

$$Q(i, j) = \frac{L(T)}{\alpha(i, j) + d(i, j)}. \quad (2)$$

The initial Q-value combines the metrics of evaluating the quality of edges in both EAX-GA and LKH, i.e., the distance and  $\alpha$ -value. The  $L(T)$  is applied to adaptively adjust the magnitude of the initial Q-value for different instances.

The Q-value can be updated by the Q-learning algorithm during the Q-LKH component of RHGA (see details in the next subsection). Note that the EAX-GA component only uses the Q-value but does not update it. Before Q-LKH terminating, the candidate set of each city in RHGA will be sorted according to the updated Q-value. Therefore, the order or the elements of the top five or top ten cities in the candidate set of each city might be changed by our reinforcement learning method. In this way, our reinforcement learning method can provide better candidate edges for both the EAX-GA and LKH components of RHGA and help the algorithm learn to select appropriate edges to be added during the  $k$ -opt process and the sub-tour merging process.

### 3.2. The Q-LKH local search algorithm

We apply the method proposed by Zheng et al. (2021) to combine Q-learning (Sutton and Barto, 1998) with LKH to learn the Q-value. The reinforced LKH algorithm (by Q-learning) is denoted as Q-LKH.

In Q-LKH, the reinforcement learning technique is combined with the core  $k$ -opt search process. A  $k$ -opt process corresponds to an episode in reinforcement learning, where the states and actions are the two endpoints of the selected edges to be added during the  $k$ -opt process. Specifically, for an episode  $(x', p) \leftarrow k\text{-opt}(x, p_1, \{p_1\}, 1)$ , the states are the cities that are going to select the edges to be added from their candidate sets, i.e., cities  $p_{2k}, k \in \{1, 2, \dots, \frac{|p|}{2} - 1\}$ , and the actions correspond to the selection of the candidate cities, i.e., cities  $p_{2k+1}, k \in \{1, 2, \dots, \frac{|p|}{2} - 1\}$ . Since the  $k$ -opt move replaces edge  $(p_{2k-1}, p_{2k})$  with edge  $(p_{2k}, p_{2k+1})$ , we define the reward of the state-action pair  $(p_{2k}, p_{2k+1})$  as  $r_k = d(p_{2k-1}, p_{2k}) - d(p_{2k}, p_{2k+1})$ , which is the benefit (the reduction in tour length) of replacing edge  $(p_{2k-1}, p_{2k})$  with edge  $(p_{2k}, p_{2k+1})$ .

The Q-LKH applies the Q-learning algorithm to update the Q-value of each state-action pair in each episode ( $k$ -opt process). For an episode

**Algorithm 4:** Q-LKH( $x_{in}, \lambda, \gamma$ )

**Input:** input solution  $x_{in}$ , learning rate:  $\lambda$ , reward discount factor:  $\gamma$   
**Output:** output solution:  $x_{out}$

- 1 Initialize the set of the cities that have not been selected as the starting city of the  $k$ -opt:  $A \leftarrow \{1, 2, \dots, n\}$ ,  $x_{out} \leftarrow x_{in}$ ;
- 2 **while** *TRUE* **do**
- 3   **if**  $A = \emptyset$  **then break**;
- 4    $p_1 \leftarrow$  a random city in  $A$ ,  $A \leftarrow A \setminus \{p_1\}$ ;
- 5    $(x_{temp}, p) \leftarrow k\text{-opt}(x_{out}, p_1, \{p_1\}, 1)$ ;
- 6   Update the Q-value of each state-action pair in  $p$  according to Eq. (3);
- 7   **if**  $l(x_{temp}) < l(x_{out})$  **then**
- 8      $x_{out} \leftarrow x_{temp}$ ;
- 9     **for**  $j \leftarrow 1 : |p|$  **do**  $A \leftarrow A \cup \{p_j\}$ ;
- 10 Sort the candidate sets of each city in descending order of the Q-value;
- 11 **return**  $x_{out}$ ;

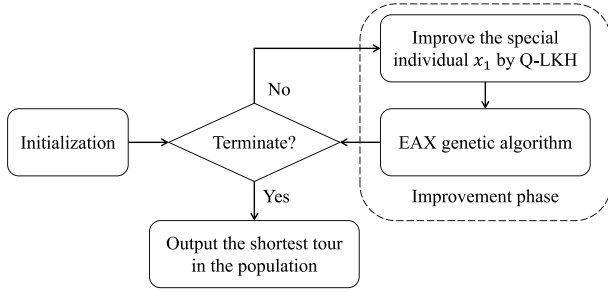


Fig. 3. Main flow of RHGA.

$(x', p) \leftarrow k\text{-opt}(x, p_1, \{p_1\}, 1)$ , the Q-value of each state-action pair  $(p_{2k}, p_{2k+1})$  is updated as follows:

$$Q(p_{2k}, p_{2k+1}) = (1 - \lambda) \cdot Q(p_{2k}, p_{2k+1}) + \lambda \cdot [r_k + \gamma \max_{a' \in CS^{p_{2k+2}}} Q(p_{2k+2}, a')], \quad (3)$$

where  $\lambda$  is the learning rate, and  $\gamma$  is the reward discount factor.

The procedure of the Q-LKH local search is presented in Algorithm 4. Q-LKH algorithm uses the  $k$ -opt heuristic (Algorithm 2) to improve the current solution  $x_{out}$  until the local optimum is reached (lines 2–9), i.e.,  $x_{out}$  cannot be improved by the  $k$ -opt heuristic starting from any starting city  $p_1 \in \{1, 2, \dots, n\}$  (line 3). Once the current solution is improved by a  $k$ -opt move (lines 7–8), each involved city can be selected as the root  $p_1$  again (line 9). Q-LKH updates the Q-value after each  $k$ -opt process (line 6), and reorders the candidate sets of each city at the end of the algorithm (line 10).

### 3.3. Main process of RHGA

The main flow of RHGA is depicted in Fig. 3, and the pseudocode of RHGA is presented in Algorithm 5. In the initialization phase of RHGA (lines 1–3), the initial candidate set of each city is generated according to the initial Q-value calculated by Eq. (2), and the initial population with  $N_{pop}$  (300 by default) individuals is generated by the *Generate\_Initial\_Pop()* function, which is a greedy 2-opt local search method used in EAX-GA (Nagata and Kobayashi, 2013). Note that the candidate sets and the Q-value are regarded as the global information in the entire RHGA algorithm.

In the improvement phase of RHGA (lines 4–25), the Q-LKH local search algorithm and the EAX Genetic Algorithm are used to improve

**Algorithm 5:** RHGA( $N_{pop}, N_{ch}, \lambda, \gamma, M_{gen}, OPT$ )

**Input:** population size:  $N_{pop}$ , number of offsprings produced by a pair of parents:  $N_{ch}$ , learning rate:  $\lambda$ , reward decay factor:  $\gamma$ , number of generations to perform Case 3:  $M_{gen}$ , length of the optimal solution:  $OPT$   
**Output:** output solution:  $x_{out}$

- 1 Generate the initial candidate sets according to the initial Q-value (Eq. (2));
- 2  $\{x_1, x_2, \dots, x_{N_{pop}}\} \leftarrow \text{Generate\_Initial\_Pop}()$ ;
- 3 Initialize  $l_{old}^1 \leftarrow +\infty$ ,  $l_{old}^{best} \leftarrow +\infty$ ,  $num \leftarrow 0$ ;
- 4 **while** a termination condition is not satisfied **do**
- 5    $x_{best} \leftarrow \arg \min_{x_i \in \{x_2, \dots, x_{N_{pop}}\}} l(x_i)$ ;
- 6   **if**  $l(x_1) = OPT \vee l(x_{best}) = OPT$  **then break**;
- 7    $num \leftarrow num + 1$ ;
- 8   **if**  $l(x_1) < l_{old}^1$  **then**
- 9      $x_1 \leftarrow \text{Q-LKH}(x_1, \lambda, \gamma)$ ;
- 10     $l_{old}^1 \leftarrow l(x_1)$ ,  $num \leftarrow 0$ ;
- 11   **if**  $l(x_{best}) < l(x_1) \wedge l(x_{best}) < l_{old}^{best}$  **then**
- 12      $l_{old}^{best} \leftarrow l(x_{best})$ ;
- 13      $x_{temp} \leftarrow \text{Q-LKH}(x_{best}, \lambda, \gamma)$ ;
- 14     **if**  $l(x_{temp}) < l(x_{best})$  **then**
- 15        $x_1 \leftarrow x_{temp}$ ,  $l_{old}^1 \leftarrow l(x_1)$ ,  $num \leftarrow 0$ ;
- 16   **if**  $num \geq M_{gen}$  **then**
- 17      $x_r \leftarrow$  a random individual in  $\{x_2, \dots, x_{N_{pop}}\}$ ;
- 18      $x_{temp} \leftarrow \text{Q-LKH}(x_r, \lambda, \gamma)$ ,  $num \leftarrow 0$ ;
- 19     **if**  $l(x_{temp}) < l(x_1)$  **then**
- 20        $x_1 \leftarrow x_{temp}$ ,  $l_{old}^1 \leftarrow l(x_1)$ ;
- 21     $rp(\cdot) \leftarrow$  a random permutation of  $\{1, 2, \dots, N_{pop}\}$ ;
- 22    **for**  $i \leftarrow 1 : N_{pop}$  **do**
- 23      $p_A \leftarrow x_{rp(i)}$ ,  $p_B \leftarrow x_{rp(i+1)}$ ;
- 24      $\{c_1, c_2, \dots, c_{N_{ch}}\} \leftarrow \text{EAX}(p_A, p_B)$ ;
- 25      $x_{rp(i)} \leftarrow \text{Select\_Survive}(c_1, \dots, c_{N_{ch}}, p_A)$ ;
- 26   **if**  $l(x_1) < l(x_{best})$  **then return**  $x_1$ ;
- 27   **else return**  $x_{best}$ ;

the population alternatively. In order to prevent the reduction of population diversity and algorithm efficiency, there is only one special individual (i.e.,  $x_1$ ) that can be improved by the Q-LKH local search algorithm. Specifically, before the procedure of the genetic algorithm (lines 21–25) at each generation, RHGA tries to improve the special individual  $x_1$  by the Q-LKH local search algorithm in the following three cases:

- **Case 1:** (lines 8–10) When  $x_1$  is just initialized or  $x_1$  was improved by EAX-GA at the last generation, i.e., when  $x_1$  may not be a local optimal solution for the Q-LKH local search algorithm. In this case, the Q-LKH algorithm will try to improve the special individual  $x_1$ .
- **Case 2:** (lines 11–15) When the tour length of the best individual  $x_{best}$  in the population other than  $x_1$  is shorter than that of  $x_1$ , and  $x_{best}$  has not been calculated by Q-LKH. In this case, the Q-LKH algorithm will try to improve  $x_{best}$ . If  $x_{best}$  can be improved, replace  $x_1$  with the improved solution, and  $x_{best}$  will not change.
- **Case 3:** (lines 16–20) When  $x_1$  has not been improved for  $M_{gen}$  generations. Note that the counter  $num$  will always be initialized to zero (no matter whether  $x_1$  can be improved). In this case, RHGA randomly selects an individual  $x_r$  in the population ( $x_r \neq x_1$ ). If Q-LKH can improve  $x_r$  and the improved tour is better than  $x_1$ , the improved tour will replace  $x_1$ , and  $x_r$  will not change.

The design of applying the Q-LKH to improve the special individual in the above three cases is reasonable and effective. Firstly, in the first two cases, the Q-LKH is prohibited from performing on its local optimal solutions to improve the algorithm's efficiency, since Q-LKH can hardly improve the local optimal solution calculated by itself. Secondly, in Case 2, the individual  $x_{best}$  with a shorter length than  $x_1$  is a very high-quality initial solution for the Q-LKH. Because  $x_{best}$  is better than the local optimal solution of  $x_1$ , and it may not be a local optimum for Q-LKH. Thus performing Q-LKH on  $x_{best}$  in Case 2 is necessary, and may obtain the near-optimal or even the optimal solution. Thirdly, in Case 3, various individuals can provide high-quality and diverse initial solutions for Q-LKH to escape from the local optima.

After the local search process in each generation, we have the EAX genetic process (lines 21–25). During this process, Each individual in the population is selected once as parent  $p_A$  and once as parent  $p_B$ , in a random order (lines 21–23). The algorithm applies the methods described in Section 2.2 to use the EAX crossover operation represented by function  $EAX()$  to parents  $p_A$  and  $p_B$  to produce  $N_{ch}$  offsprings (line 24), and then selects the surviving individual among the offsprings and  $p_A$  (line 25).

The RHGA algorithm also consists of two stages as EAX-GA does. The termination conditions of the two stages in RHGA are the same as those in EAX-GA (see Section 2.2). Moreover, if the definite optimal solution of the TSP instance is known, the input parameter  $OPT$  is set to the length of the optimal solution, otherwise zero. RHGA also terminates when the definite optimal solution is found (line 6).

### 3.4. Advantages of RHGA

This subsection illustrates the advantages of the proposed RHGA algorithm, i.e., why the RHGA is effective and better than the baseline algorithms (EAX-GA and LKH). The advantages of RHGA over the baselines include the mechanism of the hybrid genetic algorithm and the impact of reinforcement learning.

#### 3.4.1. Mechanism of the hybrid genetic algorithm

The combination of EAX-GA and LKH by the proposed mechanism can boost the performance of each other. For the EAX-GA, the special individual  $x_1$  can spread good genes (the candidate edges in LKH) to the population, and lead the population to converge to better solutions. For the LKH, the population can help the special individual  $x_1$  escape from the local optima of LKH, and provide higher-quality and more diverse initial tours than the initial tours generated by the heuristic in LKH (Helsgaun, 2000, 2009). The hybrid mechanism in RHGA can improve the baseline algorithms without reducing the population diversity and algorithm's efficiency, since there is only one special individual in the population. The experimental results also demonstrate that setting only one special individual is reasonable and efficient.

Moreover, the combination of EAX-GA and LKH can combine their advantages and overcome their disadvantages (their pros and cons are described in Section 1). That is, RHGA can solve the TSP instances with tens to hundreds of cities as well as or better than EAX-GA does, and can obtain solutions of acceptable quality within reasonable calculation time when solving the TSP instances with various scales like LKH does.

#### 3.4.2. Impact of the reinforcement learning

As indicated by the results in Helsgaun (2000), the  $\alpha$ -value outperforms the distance in determining the candidate cities or evaluating the quality of the edges. As indicated by the results in Zheng et al. (2021), the Q-value is a better choice than the  $\alpha$ -value. So why not replace the  $\alpha$ -value metric used in LKH and the distance metric used in the sub-tours merging process in EAX-GA with our learned adaptive Q-value.

In the RHGA algorithm, the reinforcement learning technique is incorporated into both the local search process and the population optimization process in RHGA, by learning an adaptive Q-value to

select and sort the candidate edges in both LKH and EAX-GA. Note that the initial candidate edges determined by the initial Q-value (Eq. (2)) are better than the candidate edges determined by the distance or  $\alpha$ -value metric (see experimental results in Section 4). Reinforcement learning can further improve the quality of the candidate edges by updating the Q-value and adjusting the candidate sets. In particular, the experimental results demonstrate that the order of the performance of EAX-GA with different metrics with decaying quality is: the adaptive Q-value (updated by Eq. (3)), the initial Q-value (Eq. (2)), the  $\alpha$ -value, and finally the distance.

## 4. Experimental results

This section presents the computational results and comparisons of RHGA, EAX-GA, LKH, and NeuroLKH (Xin et al., 2021b). The results show that RHGA significantly outperforms the other three algorithms. We first introduce the experimental setup, the benchmark instances, and the baseline algorithms, then present the experimental results.

### 4.1. Experimental setup

The experiments of RHGA were implemented in C++ and compiled by g++ with -O3 option. All the algorithms in the experiments were run on a server using an Intel® Xeon® E5-2650 v3 2.30 GHz 10-core CPU and 256 GB RAM, running Ubuntu 16.04 Linux operation system. The algorithms were all run on a single core. The parameters related to the genetic algorithm in RHGA are set to be the same as the default settings in EAX-GA (Nagata and Kobayashi, 2013), i.e.,  $N_{pop} = 300$ ,  $N_{ch} = 30$ . Other parameters are set as follows:  $\lambda = 0.1$ ,  $\gamma = 0.9$ ,  $M_{gen} = 10(\log_{10} n - 1)$  (i.e.,  $M_{gen} = 20/30/40$  when  $n = 10^3/10^4/10^5$ ). To reduce the variance in the results, we run each algorithm in the experiments 10 times on each TSP instance.

### 4.2. Benchmark instances

The RHGA algorithm was tested on all the TSP instances with the number of cities ranging from 1000 to 85,900 cities, with a total of 138, in the well-known and widely used benchmark sets for the TSP: TSPLIB,<sup>3</sup> National TSP benchmarks,<sup>4</sup> and VLSI TSP benchmarks.<sup>5</sup> Note that the number in each instance's name indicates the number of cities in that instance.

In order to make a clear comparison, we divide the 138 instances into *small* and *large* according to the instance scale. That is, an instance with less than 20,000 cities is considered to be *small*, otherwise *large*. There are a total of 111 *small* instances and 27 *large* instances among all the 138 tested instances.

Moreover, we further divide the 138 instances into the following three categories according to their difficulty:

- *Easy*: An instance is *easy* when both RHGA and EAX-GA (with the default settings) can obtain the best-known solution of this instance in each of the 10 runs (i.e., the worst solutions of RHGA and EAX-GA in 10 runs are all equal to the best-known solution when solving this instance). There are a total of 60 *easy* instances among all the 138 tested instances.
- *Medium*: An instance is *medium* when it satisfies the following two conditions: (1) the best solutions of RHGA and EAX-GA in 10 runs are all equal to the best-known solution of this instance. (2) At least one of the worst solutions of RHGA and EAX-GA is not equal to the best-known solution of this instance. There are a total of 62 *medium* instances among all the 138 tested instances.

<sup>3</sup> <http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95>.

<sup>4</sup> <http://www.math.uwaterloo.ca/tsp/world/countries.html>.

<sup>5</sup> <http://www.math.uwaterloo.ca/tsp/vlsi/index.html>.

Table 1

Comparison of RHGA and the baseline algorithms, EAX-300, EAX-400, and LKH, on 60 easy instances. Best results appear in bold.

| Instance | BKS        | RHGA                |                       |       | EAX-300             |                       |       | EAX-400             |                       |       | LKH                 |                       |       |
|----------|------------|---------------------|-----------------------|-------|---------------------|-----------------------|-------|---------------------|-----------------------|-------|---------------------|-----------------------|-------|
|          |            | Best (gap%)         | Average (gap%)        | Time  | Best (gap%)         | Average (gap%)        | Time  | Best (gap%)         | Average (gap%)        | Time  | Best (gap%)         | Average (gap%)        | Time  |
| dsj1000  | 18 660 188 | 18 660 188 (0.0000) | 18 660 188.0 (0.0000) | 35.7  | 18 660 188 (0.0000) | 18 660 188.0 (0.0000) | 20.3  | 18 660 188 (0.0000) | 18 660 188.0 (0.0000) | 46.2  | 18 660 188 (0.0000) | 18 660 188.0 (0.0000) | 70.6  |
| pr1002   | 259 045    | 259 045 (0.0000)    | 259 045.0 (0.0000)    | 27.3  | 259 045 (0.0000)    | 259 045.0 (0.0000)    | 26.9  | 259 045 (0.0000)    | 259 045.0 (0.0000)    | 41.8  | 259 045 (0.0000)    | 259 045.6 (0.0002)    | 6.1   |
| u1060    | 224 094    | 224 094 (0.0000)    | 224 094.0 (0.0000)    | 28.4  | 224 094 (0.0000)    | 224 094.0 (0.0000)    | 28.8  | 224 094 (0.0000)    | 224 094.0 (0.0000)    | 29.3  | 224 094 (0.0000)    | 224 107.5 (0.0060)    | 168.9 |
| vit1083  | 3 558      | 3 558 (0.0000)      | 3 558.0 (0.0000)      | 12.1  | 3 558 (0.0000)      | 3 558.0 (0.0000)      | 18.8  | 3 558 (0.0000)      | 3 558.0 (0.0000)      | 16.0  | 3 558 (0.0000)      | 3 558.0 (0.0000)      | 1.1   |
| xm1084   | 239 297    | 239 297 (0.0000)    | 239 297.0 (0.0000)    | 19.1  | 239 297 (0.0000)    | 239 297.0 (0.0000)    | 18.3  | 239 297 (0.0000)    | 239 297.0 (0.0000)    | 17.9  | 239 297 (0.0000)    | 239 372.6 (0.0316)    | 39.2  |
| pcb1173  | 56 892     | 56 892 (0.0000)     | 56 892.0 (0.0000)     | 21.8  | 56 892 (0.0000)     | 56 892.0 (0.0000)     | 36.2  | 56 892 (0.0000)     | 56 892.0 (0.0000)     | 38.3  | 56 892 (0.0000)     | 56 895.0 (0.0053)     | 7.4   |
| d1291    | 50 801     | 50 801 (0.0000)     | 50 801.0 (0.0000)     | 10.6  | 50 801 (0.0000)     | 50 801.0 (0.0000)     | 17.5  | 50 801 (0.0000)     | 50 801.0 (0.0000)     | 16.9  | 50 801 (0.0000)     | 50 801.0 (0.0000)     | 12.0  |
| rl1304   | 252 948    | 252 948 (0.0000)    | 252 948.0 (0.0000)    | 9.3   | 252 948 (0.0000)    | 252 948.0 (0.0000)    | 17.9  | 252 948 (0.0000)    | 252 948.0 (0.0000)    | 23.7  | 252 948 (0.0000)    | 253 156.4 (0.0824)    | 17.6  |
| rl1323   | 270 199    | 270 199 (0.0000)    | 270 199.0 (0.0000)    | 12.7  | 270 199 (0.0000)    | 270 199.0 (0.0000)    | 22.1  | 270 199 (0.0000)    | 270 199.0 (0.0000)    | 29.5  | 270 199 (0.0000)    | 270 219.6 (0.0076)    | 18.6  |
| dk1376   | 4 666      | 4 666 (0.0000)      | 4 666.0 (0.0000)      | 17.4  | 4 666 (0.0000)      | 4 666.0 (0.0000)      | 27.8  | 4 666 (0.0000)      | 4 666.0 (0.0000)      | 37.3  | 4 666 (0.0000)      | 4 666.0 (0.0000)      | 1.6   |
| nrb1379  | 56 638     | 56 638 (0.0000)     | 56 638.0 (0.0000)     | 59.9  | 56 638 (0.0000)     | 56 638.0 (0.0000)     | 52.1  | 56 638 (0.0000)     | 56 638.0 (0.0000)     | 94.3  | 56 638 (0.0000)     | 56 640.0 (0.0035)     | 14.9  |
| clka1389 | 5 085      | 5 085 (0.0000)      | 5 085.0 (0.0000)      | 31.6  | 5 085 (0.0000)      | 5 085.0 (0.0000)      | 20.6  | 5 085 (0.0000)      | 5 085.0 (0.0000)      | 45.4  | 5 085 (0.0000)      | 5 086.4 (0.0275)      | 135.4 |
| fl1400   | 20 127     | 20 127 (0.0000)     | 20 127.0 (0.0000)     | 74.2  | 20 127 (0.0000)     | 20 127.0 (0.0000)     | 28.6  | 20 127 (0.0000)     | 20 127.0 (0.0000)     | 35.9  | 20 164 (0.1838)     | 20 167.4 (0.2007)     | 703.5 |
| dja1436  | 5 257      | 5 257 (0.0000)      | 5 257.0 (0.0000)      | 25.2  | 5 257 (0.0000)      | 5 257.0 (0.0000)      | 34.1  | 5 257 (0.0000)      | 5 257.0 (0.0000)      | 30.7  | 5 257 (0.0000)      | 5 257.6 (0.0114)      | 55.7  |
| icw1483  | 4 416      | 4 416 (0.0000)      | 4 416.0 (0.0000)      | 28.5  | 4 416 (0.0000)      | 4 416.0 (0.0000)      | 33.2  | 4 416 (0.0000)      | 4 416.0 (0.0000)      | 28.3  | 4 416 (0.0000)      | 4 416.0 (0.0000)      | 19.2  |
| rl1488   | 4 264      | 4 264 (0.0000)      | 4 264.0 (0.0000)      | 18.1  | 4 264 (0.0000)      | 4 264.0 (0.0000)      | 36.7  | 4 264 (0.0000)      | 4 264.0 (0.0000)      | 46.3  | 4 264 (0.0000)      | 4 264.0 (0.0000)      | 1.2   |
| rft1583  | 5 387      | 5 387 (0.0000)      | 5 387.0 (0.0000)      | 41.8  | 5 387 (0.0000)      | 5 387.0 (0.0000)      | 41.8  | 5 387 (0.0000)      | 5 387.0 (0.0000)      | 53.8  | 5 387 (0.0000)      | 5 387.1 (0.0019)      | 23.9  |
| rw1599   | 5 533      | 5 533 (0.0000)      | 5 533.0 (0.0000)      | 37.8  | 5 533 (0.0000)      | 5 533.0 (0.0000)      | 33.8  | 5 533 (0.0000)      | 5 533.0 (0.0000)      | 62.5  | 5 533 (0.0000)      | 5 534.5 (0.0271)      | 75.5  |
| rbj1621  | 26 051     | 26 051 (0.0000)     | 26 051.0 (0.0000)     | 47.8  | 26 051 (0.0000)     | 26 051.0 (0.0000)     | 35.7  | 26 051 (0.0000)     | 26 051.0 (0.0000)     | 46.6  | 26 053 (0.0077)     | 26 076.3 (0.0971)     | 805.4 |
| d1655    | 62 128     | 62 128 (0.0000)     | 62 128.0 (0.0000)     | 47.3  | 62 128 (0.0000)     | 62 128.0 (0.0000)     | 29.7  | 62 128 (0.0000)     | 62 128.0 (0.0000)     | 57.7  | 62 128 (0.0000)     | 62 128.0 (0.0000)     | 8.3   |
| vm1748   | 336 556    | 336 556 (0.0000)    | 336 556.0 (0.0000)    | 53.3  | 336 556 (0.0000)    | 336 556.0 (0.0000)    | 43.8  | 336 556 (0.0000)    | 336 556.0 (0.0000)    | 65.4  | 336 556 (0.0000)    | 336 557.3 (0.0004)    | 20.8  |
| dj1785   | 6 115      | 6 115 (0.0000)      | 6 115.0 (0.0000)      | 61.0  | 6 115 (0.0000)      | 6 115.0 (0.0000)      | 44.4  | 6 115 (0.0000)      | 6 115.0 (0.0000)      | 78.9  | 6 115 (0.0000)      | 6 115.5 (0.0082)      | 104.1 |
| rlc1887  | 316 536    | 316 536 (0.0000)    | 316 536.0 (0.0000)    | 32.7  | 316 536 (0.0000)    | 316 536.0 (0.0000)    | 48.1  | 316 536 (0.0000)    | 316 536.0 (0.0000)    | 61.6  | 316 549 (0.0041)    | 316 549.8 (0.0044)    | 137.8 |
| dk1973   | 6 421      | 6 421 (0.0000)      | 6 421.0 (0.0000)      | 46.2  | 6 421 (0.0000)      | 6 421.0 (0.0000)      | 41.8  | 6 421 (0.0000)      | 6 421.0 (0.0000)      | 87.3  | 6 421 (0.0000)      | 6 421.0 (0.0000)      | 8.0   |
| ml1979   | 86 891     | 86 891 (0.0000)     | 86 891.0 (0.0000)     | 160.4 | 86 891 (0.0000)     | 86 891.0 (0.0000)     | 63.4  | 86 891 (0.0000)     | 86 891.0 (0.0000)     | 67.6  | 86 891 (0.0000)     | 86 892.8 (0.0021)     | 168.0 |
| dcb2086  | 6 600      | 6 600 (0.0000)      | 6 600.0 (0.0000)      | 64.7  | 6 600 (0.0000)      | 6 600.0 (0.0000)      | 69.5  | 6 600 (0.0000)      | 6 600.0 (0.0000)      | 80.5  | 6 600 (0.0000)      | 6 600.0 (0.0000)      | 36.8  |
| cl2103   | 80 450     | 80 450 (0.0000)     | 80 450.0 (0.0000)     | 48.4  | 80 450 (0.0000)     | 80 450.0 (0.0000)     | 34.8  | 80 450 (0.0000)     | 80 450.0 (0.0000)     | 51.1  | 80 454 (0.0050)     | 80 462.0 (0.0149)     | 164.6 |
| bva2114  | 6 304      | 6 304 (0.0000)      | 6 304.0 (0.0000)      | 61.0  | 6 304 (0.0000)      | 6 304.0 (0.0000)      | 71.3  | 6 304 (0.0000)      | 6 304.0 (0.0000)      | 87.4  | 6 304 (0.0000)      | 6 304.0 (0.0000)      | 3.9   |
| u1252    | 64 253     | 64 253 (0.0000)     | 64 253.0 (0.0000)     | 61.4  | 64 253 (0.0000)     | 64 253.0 (0.0000)     | 62.0  | 64 253 (0.0000)     | 64 253.0 (0.0000)     | 81.2  | 64 253 (0.0000)     | 64 287.7 (0.0540)     | 135.8 |
| qxc2175  | 6 830      | 6 830 (0.0000)      | 6 830.0 (0.0000)      | 76.6  | 6 830 (0.0000)      | 6 830.0 (0.0000)      | 75.2  | 6 830 (0.0000)      | 6 830.0 (0.0000)      | 91.0  | 6 830 (0.0000)      | 6 830.5 (0.0073)      | 106.8 |
| lyc2323  | 8 352      | 8 352 (0.0000)      | 8 352.0 (0.0000)      | 49.2  | 8 352 (0.0000)      | 8 352.0 (0.0000)      | 52.8  | 8 352 (0.0000)      | 8 352.0 (0.0000)      | 45.1  | 8 352 (0.0000)      | 8 353.5 (0.0180)      | 86.8  |
| dea2382  | 8 017      | 8 017 (0.0000)      | 8 017.0 (0.0000)      | 71.6  | 8 017 (0.0000)      | 8 017.0 (0.0000)      | 78.4  | 8 017 (0.0000)      | 8 017.0 (0.0000)      | 62.2  | 8 017 (0.0000)      | 8 017.3 (0.0037)      | 130.0 |
| pr2392   | 378 032    | 378 032 (0.0000)    | 378 032.0 (0.0000)    | 39.1  | 378 032 (0.0000)    | 378 032.0 (0.0000)    | 82.7  | 378 032 (0.0000)    | 378 032.0 (0.0000)    | 69.3  | 378 032 (0.0000)    | 378 032.0 (0.0000)    | 0.6   |
| rbw2481  | 7 724      | 7 724 (0.0000)      | 7 724.0 (0.0000)      | 59.3  | 7 724 (0.0000)      | 7 724.0 (0.0000)      | 67.3  | 7 724 (0.0000)      | 7 724.1 (0.0013)      | 99.3  | 7 724 (0.0000)      | 7 724.0 (0.0000)      | 3.9   |
| pds2566  | 7 643      | 7 643 (0.0000)      | 7 643.0 (0.0000)      | 102.6 | 7 643 (0.0000)      | 7 643.0 (0.0000)      | 89.4  | 7 643 (0.0000)      | 7 643.0 (0.0000)      | 86.6  | 7 643 (0.0000)      | 7 643.0 (0.0000)      | 68.3  |
| mlr2597  | 8 071      | 8 071 (0.0000)      | 8 071.0 (0.0000)      | 47.6  | 8 071 (0.0000)      | 8 071.0 (0.0000)      | 91.0  | 8 071 (0.0000)      | 8 071.0 (0.0000)      | 117.3 | 8 071 (0.0000)      | 8 071.0 (0.0000)      | 16.6  |
| lrw2802  | 8 423      | 8 423 (0.0000)      | 8 423.0 (0.0000)      | 86.1  | 8 423 (0.0000)      | 8 423.0 (0.0000)      | 97.9  | 8 423 (0.0000)      | 8 423.0 (0.0000)      | 127.7 | 8 423 (0.0000)      | 8 424.2 (0.0142)      | 184.9 |
| lsm2854  | 8 014      | 8 014 (0.0000)      | 8 014.0 (0.0000)      | 103.2 | 8 014 (0.0000)      | 8 014.0 (0.0000)      | 104.6 | 8 014 (0.0000)      | 8 014.0 (0.0000)      | 137.5 | 8 014 (0.0000)      | 8 014.0 (0.0000)      | 103.5 |
| xva2993  | 8 492      | 8 492 (0.0000)      | 8 492.0 (0.0000)      | 128.1 | 8 492 (0.0000)      | 8 492.0 (0.0000)      | 114.5 | 8 492 (0.0000)      | 8 492.0 (0.0000)      | 154.9 | 8 492 (0.0000)      | 8 493.3 (0.0153)      | 316.0 |
| pcb3038  | 137 694    | 137 694 (0.0000)    | 137 694.0 (0.0000)    | 151.1 | 137 694 (0.0000)    | 137 694.0 (0.0000)    | 162.1 | 137 694 (0.0000)    | 137 694.0 (0.0000)    | 172.2 | 137 694 (0.0000)    | 137 701.2 (0.0052)    | 86.5  |
| dke3097  | 10 539     | 10 539 (0.0000)     | 10 539.0 (0.0000)     | 127.4 | 10 539 (0.0000)     | 10 539.0 (0.0000)     | 91.5  | 10 539 (0.0000)     | 10 539.0 (0.0000)     | 107.2 | 10 539 (0.0000)     | 10 539.1 (0.0009)     | 154.5 |
| lsm3119  | 9 114      | 9 114 (0.0000)      | 9 114.0 (0.0000)      | 120.3 | 9 114 (0.0000)      | 9 114.0 (0.0000)      | 132.5 | 9 114 (0.0000)      | 9 114.0 (0.0000)      | 156.9 | 9 114 (0.0000)      | 9 114.4 (0.0044)      | 112.9 |
| lta3140  | 9 517      | 9 517 (0.0000)      | 9 517.0 (0.0000)      | 134.5 | 9 517 (0.0000)      | 9 517.0 (0.0000)      | 136.6 | 9 517 (0.0000)      | 9 517.1 (0.0011)      | 167.8 | 9 517 (0.0000)      | 9 517.7 (0.0074)      | 183.5 |
| dhb3386  | 11 137     | 11 137 (0.0000)     | 11 137.0 (0.0000)     | 127.5 | 11 137 (0.0000)     | 11 137.0 (0.0000)     | 154.8 | 11 137 (0.0000)     | 11 137.0 (0.0000)     | 183.3 | 11 137 (0.0000)     | 11 137.0 (0.0000)     | 101.1 |
| pr3672   | 9 601      | 9 601 (0.0000)      | 9 601.0 (0.0000)      | 158.2 | 9 601 (0.0000)      | 9 601.0 (0.0000)      | 157.0 | 9 601 (0.0000)      | 9 601.0 (0.0000)      | 147.0 | 9 601 (0.0000)      | 9 601.0 (0.0000)      | 107.2 |
| lth3729  | 11 821     | 11 821 (0.0000)     | 11 821.0 (0.0000)     | 171.6 | 11 821 (0.0000)     | 11 821.0 (0.0000)     | 165.4 | 11 821 (0.0000)     | 11 821.0 (0.0000)     | 237.5 | 11 821 (0.0000)     | 11 822.2 (0.0102)     | 716.8 |
| xbs3937  | 11 239     | 11 239 (0.0000)     | 11 239.0 (0.0000)     | 134.8 | 11 239 (0.0000)     | 11 239.0 (0.0000)     | 147.5 | 11 239 (0.0000)     | 11 239.0 (0.0000)     | 197.1 | 11 239 (0.0000)     | 11 240.4 (0.0125)     | 639.4 |
| bgd4355  | 12 723     | 12 723 (0.0000)     | 12 723.0 (0.0000)     | 238.1 | 12 723 (0.0000)     | 12 723.0 (0.0000)     | 176.9 | 12 723 (0.0000)     | 12 723.0 (0.0000)     | 224.7 | 12 723 (0.0000)     | 12 728.0 (0.0393)     | 582.5 |
| bgd4396  | 13 009     | 13 009 (0.0000)     | 13 009.0 (0.0000)     | 205.0 | 13 009 (0.0000)     | 13 009.0 (0.0000)     | 145.3 | 13 009 (0.0000)     | 13 009.1 (0.0008)     | 216.1 | 13 009 (0.0000)     | 13 010.3 (0.0100)     | 388.3 |
| frv4410  | 10 711     | 10 711 (0.0000)     | 10 711.0 (0.0000)     | 224.7 | 10 711 (0.0000)     | 10 711.0 (0.0000)     | 201.4 | 10 711 (0.0000)     | 10 711.0 (0.0000)     | 266.7 | 10 711 (0.0000)     | 10 712.1 (0.0103)     | 356.0 |
| fln4461  | 182 566    | 182 566 (0.0000)    | 182 566.0 (0.0000)    | 497.9 | 182 566 (0.0000)    | 182 566.0 (0.0000)    | 628.5 | 182 566 (0.0000)    | 182 566.4 (0.0002)    | 583.6 | 182 566 (0.0000)    | 182 566.5 (0.0003)    | 3     |



Table 2

Comparison of RHGA and the baseline algorithms, EAX-300, EAX-400, and LKH, on 62 *medium* instances. Best results appear in bold.

| Instance  | BKS        | RHGA                       |                       |        | EAX-300                    |                       |          | EAX-400                    |                       |        | LKH                        |                       |        |
|-----------|------------|----------------------------|-----------------------|--------|----------------------------|-----------------------|----------|----------------------------|-----------------------|--------|----------------------------|-----------------------|--------|
|           |            | Best (gap%)                | Average (gap%)        | Time   | Best (gap%)                | Average (gap%)        | Time     | Best (gap%)                | Average (gap%)        | Time   | Best (gap%)                | Average (gap%)        | Time   |
| si1032    | 92 650     | <b>92 650 (0.0000)</b>     | 92 650.1 (0.0001)     | 8.1    | <b>92 650 (0.0000)</b>     | 92 650.5 (0.0005)     | 1.6      | <b>92 650 (0.0000)</b>     | 92 650.1 (0.0001)     | 3.7    | <b>92 650 (0.0000)</b>     | 92 650.0 (0.0000)     | 34.4   |
| u1432     | 152 970    | <b>152 970 (0.0000)</b>    | 152 970.0 (0.0000)    | 25.7   | <b>152 970 (0.0000)</b>    | 152 973.6 (0.0024)    | 38.3     | <b>152 970 (0.0000)</b>    | 152 979.0 (0.0059)    | 56.3   | <b>152 970 (0.0000)</b>    | 152 970.0 (0.0000)    | 0.9    |
| fi1577    | 22 249     | <b>22 249 (0.0000)</b>     | 22 249.0 (0.0000)     | 67.6   | <b>22 249 (0.0000)</b>     | 22 309.5 (0.2719)     | 38.9     | <b>22 249 (0.0000)</b>     | 22 322.9 (0.3321)     | 34.3   | 22 261 (0.0539)            | 22 262.1 (0.0589)     | 765.1  |
| fnh1615   | 4 956      | <b>4 956 (0.0000)</b>      | 4 956.1 (0.0020)      | 48.4   | <b>4 956 (0.0000)</b>      | 4 956.6 (0.0121)      | 35.9     | <b>4 956 (0.0000)</b>      | 4 956.7 (0.0141)      | 77.6   | <b>4 956 (0.0000)</b>      | 4 956.0 (0.0000)      | 36.5   |
| u1817     | 57 201     | <b>57 201 (0.0000)</b>     | 57 209.1 (0.0142)     | 51.0   | <b>57 201 (0.0000)</b>     | 57 216.0 (0.0262)     | 67.5     | <b>57 201 (0.0000)</b>     | 57 216.6 (0.0273)     | 55.5   | <b>57 201 (0.0000)</b>     | 57 251.1 (0.0876)     | 93.7   |
| dcc1911   | 6 396      | <b>6 396 (0.0000)</b>      | 6 396.0 (0.0000)      | 53.7   | <b>6 396 (0.0000)</b>      | 6 396.1 (0.0016)      | 65.6     | <b>6 396 (0.0000)</b>      | 6 396.0 (0.0000)      | 67.5   | <b>6 396 (0.0000)</b>      | 6 397.0 (0.0156)      | 173.6  |
| djb2036   | 6 197      | <b>6 197 (0.0000)</b>      | 6 197.0 (0.0000)      | 55.5   | <b>6 197 (0.0000)</b>      | 6 197.2 (0.0032)      | 76.6     | <b>6 197 (0.0000)</b>      | 6 197.0 (0.0000)      | 86.0   | <b>6 197 (0.0000)</b>      | 6 197.1 (0.0016)      | 26.0   |
| bck2217   | 6 764      | <b>6 764 (0.0000)</b>      | 6 764.3 (0.0044)      | 81.9   | <b>6 764 (0.0000)</b>      | 6 764.7 (0.0103)      | 104.3    | <b>6 764 (0.0000)</b>      | 6 764.2 (0.0030)      | 121.4  | 6 765 (0.0148)             | 6 765.2 (0.0177)      | 221.2  |
| xpr2308   | 7 219      | <b>7 219 (0.0000)</b>      | 7 219.1 (0.0014)      | 80.4   | <b>7 219 (0.0000)</b>      | 7 219.2 (0.0028)      | 98.4     | <b>7 219 (0.0000)</b>      | 7 219.0 (0.0000)      | 68.8   | <b>7 219 (0.0000)</b>      | 7 219.2 (0.0028)      | 44.3   |
| bcb2762   | 8 234      | <b>8 234 (0.0000)</b>      | 8 234.1 (0.0012)      | 131.6  | <b>8 234 (0.0000)</b>      | 8 234.4 (0.0049)      | 129.8    | <b>8 234 (0.0000)</b>      | 8 234.0 (0.0000)      | 132.6  | <b>8 234 (0.0000)</b>      | 8 235.1 (0.0134)      | 249.9  |
| djb2924   | 10 128     | <b>10 128 (0.0000)</b>     | 10 128.0 (0.0000)     | 127.1  | <b>10 128 (0.0000)</b>     | 10 128.1 (0.0010)     | 127.9    | <b>10 128 (0.0000)</b>     | 10 128.0 (0.0000)     | 99.1   | <b>10 128 (0.0000)</b>     | 10 128.1 (0.0010)     | 120.0  |
| pia3056   | 8 258      | <b>8 258 (0.0000)</b>      | 8 258.5 (0.0061)      | 156.4  | <b>8 258 (0.0000)</b>      | 8 258.8 (0.0097)      | 141.6    | <b>8 258 (0.0000)</b>      | 8 258.6 (0.0073)      | 157.1  | <b>8 258 (0.0000)</b>      | 8 262.2 (0.0509)      | 526.0  |
| fdp3256   | 10 008     | <b>10 008 (0.0000)</b>     | 10 008.1 (0.0010)     | 127.4  | <b>10 008 (0.0000)</b>     | 10 008.2 (0.0020)     | 157.8    | <b>10 008 (0.0000)</b>     | 10 008.5 (0.0050)     | 206.2  | 10 009 (0.0100)            | 10 009.7 (0.0170)     | 450.9  |
| beg3293   | 9 772      | <b>9 772 (0.0000)</b>      | 9 772.2 (0.0020)      | 131.1  | <b>9 772 (0.0000)</b>      | 9 772.0 (0.0000)      | 142.3    | <b>9 772 (0.0000)</b>      | 9 772.1 (0.0010)      | 182.6  | <b>9 772 (0.0000)</b>      | 9 772.2 (0.0020)      | 171.3  |
| ns0496    | 96 132     | <b>96 132 (0.0000)</b>     | 96 132.1 (0.0001)     | 169.1  | <b>96 132 (0.0000)</b>     | 96 132.0 (0.0000)     | 150.8    | <b>96 132 (0.0000)</b>     | 96 132.0 (0.0000)     | 189.0  | 96 180 (0.0499)            | 96 201.4 (0.0722)     | 1749.1 |
| flj3649   | 9 272      | <b>9 272 (0.0000)</b>      | 9 272.0 (0.0000)      | 176.3  | <b>9 272 (0.0000)</b>      | 9 272.5 (0.0054)      | 167.4    | <b>9 272 (0.0000)</b>      | 9 272.0 (0.0000)      | 189.4  | <b>9 272 (0.0000)</b>      | 9 272.0 (0.0000)      | 88.5   |
| dlb3694   | 10 959     | <b>10 959 (0.0000)</b>     | 10 959.3 (0.0027)     | 216.7  | <b>10 959 (0.0000)</b>     | 10 959.3 (0.0027)     | 185.4    | <b>10 959 (0.0000)</b>     | 10 959.1 (0.0009)     | 184.0  | <b>10 959 (0.0000)</b>     | 10 959.7 (0.0064)     | 334.4  |
| xqc3891   | 11 995     | <b>11 995 (0.0000)</b>     | 11 996.1 (0.0092)     | 228.8  | <b>11 995 (0.0000)</b>     | 11 996.0 (0.0083)     | 245.7    | <b>11 995 (0.0000)</b>     | 11 995.9 (0.0075)     | 347.4  | <b>11 995 (0.0000)</b>     | 11 998.2 (0.0267)     | 454.0  |
| dkc3938   | 12 503     | <b>12 503 (0.0000)</b>     | 12 503.0 (0.0000)     | 187.2  | <b>12 503 (0.0000)</b>     | 12 503.1 (0.0008)     | 140.9    | <b>12 503 (0.0000)</b>     | 12 503.0 (0.0000)     | 236.5  | <b>12 503 (0.0000)</b>     | 12 503.8 (0.0064)     | 697.1  |
| dkf3954   | 12 538     | <b>12 538 (0.0000)</b>     | 12 538.0 (0.0000)     | 180.3  | <b>12 538 (0.0000)</b>     | 12 538.2 (0.0016)     | 124.4    | <b>12 538 (0.0000)</b>     | 12 538.0 (0.0000)     | 227.3  | <b>12 538 (0.0000)</b>     | 12 538.6 (0.0048)     | 113.9  |
| fqm5087   | 13 029     | <b>13 029 (0.0000)</b>     | 13 029.0 (0.0000)     | 428.4  | <b>13 029 (0.0000)</b>     | 13 029.1 (0.0008)     | 337.7    | <b>13 029 (0.0000)</b>     | 13 029.1 (0.0008)     | 400.0  | <b>13 029 (0.0000)</b>     | 13 029.5 (0.0038)     | 1790.7 |
| rl5934    | 556 045    | <b>556 045 (0.0000)</b>    | 556 072.3 (0.0049)    | 476.1  | <b>556 045 (0.0000)</b>    | 556 090.9 (0.0083)    | 311.0    | <b>556 045 (0.0000)</b>    | 556 054.1 (0.0016)    | 474.4  | 556 136 (0.0164)           | 556 309.8 (0.0476)    | 734.2  |
| z6117     | 394 718    | <b>394 718 (0.0000)</b>    | 394 721.2 (0.0009)    | 694.0  | <b>394 718 (0.0000)</b>    | 394 721.6 (0.0009)    | 572.0    | <b>394 718 (0.0000)</b>    | 394 721.4 (0.0009)    | 829.8  | 394 726 (0.0020)           | 394 747.6 (0.0075)    | 3068.4 |
| xsc6880   | 21 535     | <b>21 535 (0.0000)</b>     | 21 535.1 (0.0005)     | 641.7  | <b>21 535 (0.0000)</b>     | 21 535.6 (0.0028)     | 454.3    | <b>21 535 (0.0000)</b>     | 21 535.1 (0.0005)     | 762.6  | 21 537 (0.0093)            | 21 540.6 (0.0260)     | 1743.2 |
| q7146     | 172 386    | <b>172 386 (0.0000)</b>    | 172 386.0 (0.0000)    | 1199.0 | <b>172 386 (0.0000)</b>    | 172 386.2 (0.0001)    | 1058.4   | <b>172 386 (0.0000)</b>    | 172 386.0 (0.0000)    | 981.6  | 172 378 (0.2042)           | 172 738.7 (0.2046)    | 2068.6 |
| flp7454   | 19 535     | <b>19 535 (0.0000)</b>     | 19 535.0 (0.0000)     | 457.2  | <b>19 535 (0.0000)</b>     | 19 535.1 (0.0005)     | 498.9    | <b>19 535 (0.0000)</b>     | 19 535.0 (0.0000)     | 496.2  | <b>19 535 (0.0000)</b>     | 19 537.1 (0.0107)     | 1283.5 |
| pms8079   | 114 855    | <b>114 855 (0.0000)</b>    | 114 855.3 (0.0003)    | 1301.2 | <b>114 855 (0.0000)</b>    | 114 854.9 (0.0260)    | 1550.1   | <b>114 855 (0.0000)</b>    | 114 884.0 (0.0252)    | 2081.2 | 114 872 (0.0148)           | 114 893.9 (0.0339)    | 4044.8 |
| ida8197   | 22 338     | <b>22 338 (0.0000)</b>     | 22 338.1 (0.0004)     | 424.8  | <b>22 338 (0.0000)</b>     | 22 338.2 (0.0009)     | 501.7    | <b>22 338 (0.0000)</b>     | 22 338.0 (0.0000)     | 740.8  | <b>22 338 (0.0000)</b>     | 22 339.2 (0.0054)     | 930.7  |
| ic8246    | 206 171    | <b>206 171 (0.0000)</b>    | 206 171.6 (0.0003)    | 1346.3 | <b>206 171 (0.0000)</b>    | 206 172.7 (0.0008)    | 1295.8   | <b>206 171 (0.0000)</b>    | 206 171.0 (0.0000)    | 1943.4 | <b>206 171 (0.0000)</b>    | 206 175.2 (0.0020)    | 1213.8 |
| ar9152    | 837 479    | <b>837 479 (0.0000)</b>    | 837 479.0 (0.0000)    | 1285.9 | <b>837 479 (0.0000)</b>    | 837 528.0 (0.0059)    | 1202.9   | <b>837 479 (0.0000)</b>    | 837 554.2 (0.0090)    | 1696.5 | 837 575 (0.0115)           | 837 641.8 (0.0194)    | 4579.3 |
| ja9847    | 491 924    | <b>491 924 (0.0000)</b>    | 491 924.4 (0.0003)    | 224.3  | <b>491 924 (0.0000)</b>    | 491 925.4 (0.0007)    | 96.1     | <b>491 924 (0.0000)</b>    | 491 926.6 (0.0005)    | 1430.9 | 491 947 (0.0047)           | 492 073.2 (0.0343)    | 2859.2 |
| gr9882    | 300 899    | <b>300 899 (0.0000)</b>    | 300 900.8 (0.0006)    | 136.2  | <b>300 899 (0.0000)</b>    | 300 901.6 (0.0009)    | 1316.7   | <b>300 899 (0.0000)</b>    | 300 900.4 (0.0005)    | 1396.9 | <b>300 899 (0.0000)</b>    | 300 901.0 (0.0007)    | 1457.0 |
| kz9976    | 1 061 881  | <b>1 061 881 (0.0000)</b>  | 1 061 881.5 (0.0000)  | 1958.3 | <b>1 061 881 (0.0000)</b>  | 1 061 882.0 (0.0001)  | 1745.1   | <b>1 061 881 (0.0000)</b>  | 1 061 881.0 (0.0000)  | 1725.6 | <b>1 061 881 (0.0000)</b>  | 1 061 941.2 (0.0057)  | 2113.4 |
| xmc10150  | 28 387     | <b>28 387 (0.0000)</b>     | 28 387.1 (0.0004)     | 988.6  | <b>28 387 (0.0000)</b>     | 28 387.3 (0.0011)     | 688.0    | <b>28 387 (0.0000)</b>     | 28 387.3 (0.0011)     | 951.5  | <b>28 387 (0.0000)</b>     | 28 389.3 (0.0081)     | 2025.7 |
| f10 639   | 520 527    | <b>520 527 (0.0000)</b>    | 520 527.1 (0.0000)    | 2336.6 | <b>520 527 (0.0000)</b>    | 520 527.3 (0.0001)    | 1847.9   | <b>520 527 (0.0000)</b>    | 520 527.1 (0.0000)    | 2340.6 | <b>520 531 (0.0008)</b>    | 520 561.8 (0.0067)    | 3260.7 |
| rl1 849   | 923 288    | <b>923 288 (0.0000)</b>    | 923 288.0 (0.0000)    | 2383.0 | <b>923 288 (0.0000)</b>    | 923 291.6 (0.0004)    | 1729.9   | <b>923 288 (0.0000)</b>    | 923 288.0 (0.0000)    | 2173.4 | <b>923 288 (0.0000)</b>    | 923 362.7 (0.0081)    | 3684.3 |
| usa13 509 | 19 982 859 | <b>19 982 859 (0.0000)</b> | 19 982 881.7 (0.0001) | 5056.8 | <b>19 982 859 (0.0000)</b> | 19 982 894.6 (0.0002) | 3357.8   | <b>19 982 859 (0.0000)</b> | 19 982 859.0 (0.0000) | 4688.6 | <b>19 982 859 (0.0000)</b> | 19 983 103.4 (0.0012) | 5146.7 |
| mx13584   | 37 083     | <b>37 083 (0.0000)</b>     | 37 083.2 (0.0005)     | 1821.8 | <b>37 083 (0.0000)</b>     | 37 083.2 (0.0005)     | 1691.1   | <b>37 083 (0.0000)</b>     | 37 083.0 (0.0000)     | 2179.5 | <b>37 083 (0.0000)</b>     | 37 088.7 (0.0154)     | 4700.7 |
| vol14 185 | 427 377    | <b>427 377 (0.0000)</b>    | 427 377.6 (0.0001)    | 4215.7 | <b>427 377 (0.0000)</b>    | 427 378.2 (0.0003)    | 2643.3   | <b>427 377 (0.0000)</b>    | 427 377.2 (0.0000)    | 4121.8 | 427 382 (0.0012)           | 427 399.5 (0.0053)    | 6264.8 |
| hxl14233  | 45 462     | <b>45 462 (0.0000)</b>     | 45 463.3 (0.0029)     | 2122.4 | <b>45 462 (0.0000)</b>     | 45 463.6 (0.0035)     | 1518.4   | <b>45 462 (0.0000)</b>     | 45 463.6 (0.0035)     | 1988.5 | 45 464 (0.0044)            | 45 468.5 (0.0143)     | 5723.8 |
| ho14473   | 177 092    | <b>177 092 (0.0000)</b>    | 177 092.2 (0.0001)    | 6841.0 | <b>177 092 (0.0000)</b>    | 177 092.4 (0.0002)    | 4906.4   | <b>177 092 (0.0000)</b>    | 177 092.3 (0.0002)    | 5930.9 | 177 235 (0.0807)           | 177 303.2 (0.1193)    | 7339.7 |
| d15112    | 1 573 084  | <b>1 573 084 (0.0000)</b>  | 1 573 084.4 (0.0000)  | 6941.3 | <b>1 573 084 (0.0000)</b>  | 1 573 084.8 (0.0001)  | 5947.6   | <b>1 573 084 (0.0000)</b>  | 1 573 084.6 (0.0000)  | 8157.7 | 1 573 085 (0.0001)         | 1 573 146.9 (0.0040)  | 7257.4 |
| it16862   | 557 315    | <b>557 315 (0.0000)</b>    | 557 316.9 (0.0003)    | 7958.8 | <b>557 315 (0.0000)</b>    | 557 319.3 (0.0008)    | 3518.9   | <b>557 315 (0.0000)</b>    | 557 317.6 (0.0005)    | 5965.3 | 557 321 (0.0011)           | 557 343.0 (0.0050)    | 8436.6 |
| ajal16928 | 52 850     | <b>52 850 (0.0000)</b>     | 52 850.2 (0.0004)     | 3124.1 | <b>52 850 (0.0000)</b>     | 52 850.7 (0.0013)     | 1897.0   | <b>52 850 (0.0000)</b>     | 52 850.7 (0.0013)     | 3283.7 | <b>52 850 (0.0000)</b>     | 52 856.3 (0.0119)     | 8175.8 |
| pjh17 845 | 48 092     | <b>48 092 (0.0000)</b>     | 48 092.5 (0.0010)     | 3292.7 | <b>48 092 (0.0000)</b>     | 48 092.6 (0.0004)     | 2449.1   | <b>48 092 (0.0000)</b>     | 48 092.6 (0.0012)     | 3608.3 | 48 094 (0.0042)            | 48 100.6 (0.0179)     | 8769.8 |
| d18512    | 645 238    | <b>645 238 (0.0000)</b>    | 645 238.6 (0.0001)    | 9809.9 | <b>645 238 (0.0000)</b>    | 645 238.7 (0.0001)    | 10 085.7 | <b>645 238 (0.0000)</b> </ |                       |        |                            |                       |        |

**Table 4**  
Summarized comparisons between RHGA and EAX-300, EAX-400, and LKH.

| Algorithm pair   | Indicator | #Wins | #Ties | #Losses |
|------------------|-----------|-------|-------|---------|
| RHGA vs. EAX-300 | Best      | 13    | 125   | 0       |
|                  | Average   | 67    | 65    | 6       |
| RHGA vs. EAX-400 | Best      | 9     | 127   | 2       |
|                  | Average   | 40    | 72    | 26      |
| RHGA vs. LKH     | Best      | 54    | 84    | 0       |
|                  | Average   | 114   | 21    | 3       |

for which RHGA obtains a better, equal, and worse result than the compared algorithm according to the best solution and average solution indicators.

From the results in Tables 1, 2, 3, and 4, we observe that:

(1) On all the 60 *easy* instances, RHGA, EAX-300, and EAX-400 can easily yield the optimal solution in almost each of the 10 runs.

(2) On all the 62 *medium* instances, RHGA exhibits better stability and robustness than EAX-300 and EAX-400, such as in solving the instances *u1432*, *fl1577*, *pm8079*, and *ar9152*. The average gap of the average solutions of RHGA is 84.5% (85.1%) less than that of EAX-300 (EAX-400).

(3) On all the 16 *hard* instances, RHGA greatly outperforms EAX-300 and EAX-400. Specifically, the best solutions, the average solutions, and the worst solutions of RHGA are all better than those of EAX-300. The best solutions of RHGA are better than those of EAX-400 in 9 *hard* instances, and worse than those of EAX-400 in 2 *hard* instances. The average gap of the best solutions of RHGA is 96.4% (84.6%) less than that of EAX-300 (EAX-400), and the average gap of the average solutions of RHGA is 84.0% (82.4%) less than that of EAX-300 (EAX-400), indicating a significant improvement.

(4) The calculating time of RHGA and EAX-400 is close, indicating that RHGA can yield better performance than EAX-GA within the same parameters (compared to EAX-300) or similar calculation time (compared to EAX-400).

(5) The LKH is weaker than the other three algorithms in solving most of the tested instances. However, in solving the instances such as *u2319* and *pla7397*, LKH shows significantly better performance. Thus EAX-GA and LKH are complementary in solving different TSP instances, and our combination can make full use of their advantages and boost their performance.

To make a clearer comparison of RHGA and EAX-GA, we apply the cumulative metrics including the cumulative gap on the solution quality and cumulative run time to compare RHGA with EAX-300 and EAX-400 in solving all the 51 *small* but not *easy* instances and all the 27 *large* instances. Let  $gap(j) = \frac{1}{10} \sum_{i=1}^{10} \frac{A_i - BKS_j}{BKS_j}$  be the average gap of calculating the  $j$ th instance by an algorithm in 10 runs, where  $A_i$  is the result of the  $i$ th calculation and  $BKS_j$  is the best-known solution of the  $j$ th instance. The smaller the average gap, the closer the average solution is to the best-known solution. For an algorithm,  $C_{gap}(j) = \sum_{i=1}^j gap(i)$  is the cumulative gap. The cumulative run time can be calculated similarly. The comparison results are shown in Fig. 4.

The results indicate again that the robustness and stability of EAX-GA are not good. It shows much worse performance than RHGA in solving some instances, such as *fl1577*, *u2319*, *fl3795*, *pla33810*, and *pla85900*. The increase of population size from 300 to 400 cannot help EAX-GA escape from the local optima when solving these instances, while the proposed methods including the hybrid mechanism and reinforcement learning could. We can also observe that the calculation time of RHGA is close to EAX-400 and a little bit longer than that of EAX-300. The results clearly show again that RHGA significantly outperforms EAX-GA within the same parameters or similar calculation time.

#### 4.5. Comparing RHGA with NeuroLKH

We then compare RHGA with NeuroLKH\_R and NeuroLKH\_M, in solving all the instances with the number of cities ranging from 1000 to 10,000 and two-dimensional Euclidean distance (EUC\_2D) metric (NeuroLKH only supports the EUC\_2D metric), a total of 92. We extract the instances that RHGA, NeuroLKH\_R, and NeuroLKH\_M can always yield the optimal solution in each of the 10 runs. The results of the remaining 77 instances are shown in Table 5. We compare the best and average solutions in 10 runs obtained by the algorithms.

From the results in Table 5, we can observe that:

(1) RHGA significantly outperforms NeuroLKH\_R and NeuroLKH\_M. RHGA can yield all the best-known solutions in 10 runs. The best solutions of RHGA are better than those of NeuroLKH\_R (NeuroLKH\_M) in 42 (29) instances. The average solutions of RHGA are better than those of NeuroLKH\_R (NeuroLKH\_M) on 72 (60) instances. The average gaps of the best solutions and average solutions of RHGA are much smaller than those of NeuroLKH\_R and NeuroLKH\_M. The average calculation time of RHGA is also much smaller than that of NeuroLKH\_R and NeuroLKH\_M.

(2) The performance of NeuroLKH\_M is better than that of NeuroLKH\_R, indicating that the performance of NeuroLKH relies on the structure of the training instances. Generating reasonable training instances that help NeuroLKH work well on instances with diverse structures is challenging. Moreover, both NeuroLKH\_R and NeuroLKH\_M are not good at solving large instances, indicating that the bottlenecks in large scale instances still limit algorithms based on deep neural networks.

Actually, NeuroLKH (Xin et al., 2021b) mainly improves LKH in solving instances with the same structure as its training instances, e.g., instances with uniformly distributed nodes. For large public instances (e.g., public instances with more than 6000 cities) with various structures, NeuroLKH usually shows similar or worse performance than LKH does, indicating again the scale and performance limitation of deep learning based methods.

#### 5. Further analysis

This section provides insight on why and how the proposed RHGA is effective, suggesting that the creative combination of EAX-GA and LKH can boost the performance mutually. The results further indicate that the adaptive Q-value learned by reinforcement learning is a better metric for determining the candidate cities and evaluating the edge quality than the  $\alpha$ -value in LKH as well as the distance in EAX-GA. We first introduce various variant algorithms involved in the experiments, then present and analyze the results.

##### 5.1. Various variants of the algorithm

We first present various variant algorithms of RHGA and EAX-GA for comparison and analysis. The variant algorithms include the following:

- **Alpha-EAX:** A variant of EAX-300 using the  $\alpha$ -value (Eq. (1)) to replace the distance metric used in the process of merging sub-tours in EAX-300 (see Step 5 in Section 2.2).
- **FixQ-EAX:** A variant of EAX-300 using the initial Q-value (Eq. (2)) to replace the distance metric used in the process of merging sub-tours in EAX-300.
- **Q-EAX:** A variant of EAX-300 using the adaptive Q-value learned by the Q-learning method (Eq. (3)) to replace the distance metric used in the process of merging sub-tours in EAX-300. The population size in Q-EAX is set to be 301. The extra individual is the special individual used to learn the Q-value and provide the adaptive Q-value for the algorithm. The special individual in Q-EAX can be improved by the Q-LKH local search and the genetic algorithm, but other individuals cannot be improved by crossover

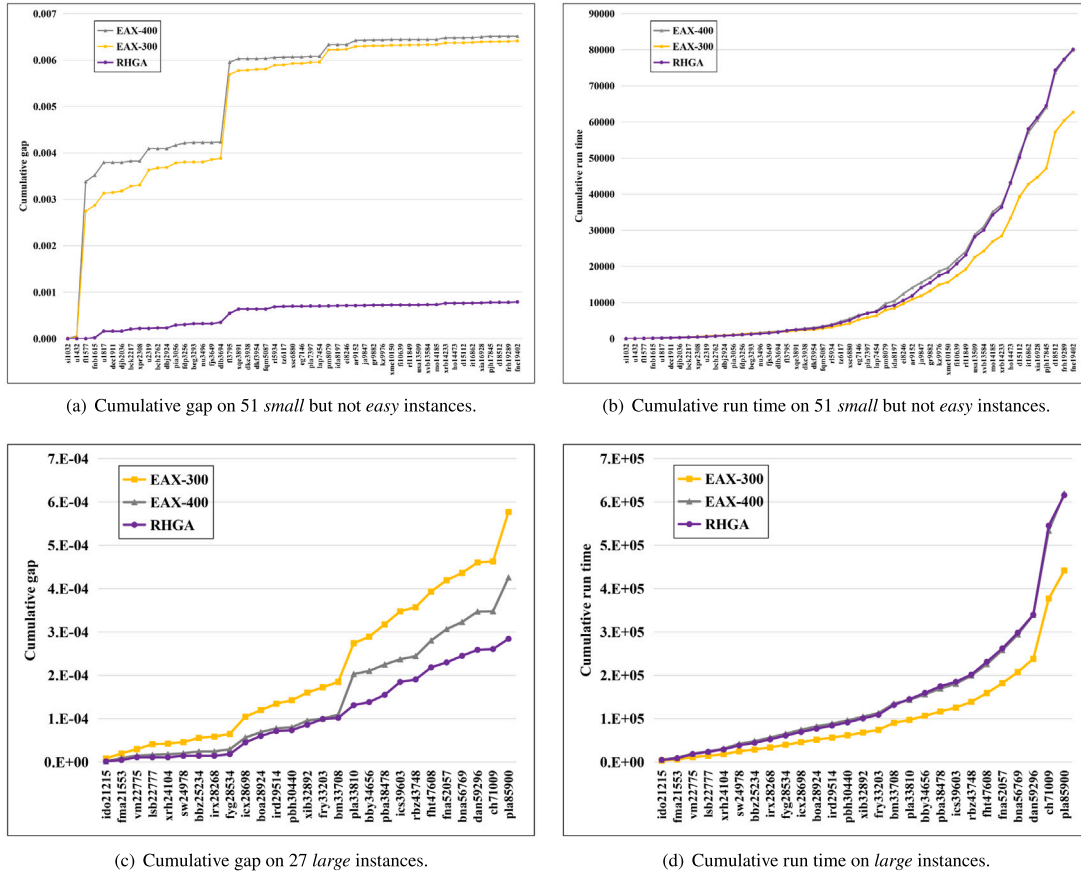


Fig. 4. Comparison results of RHGA, EAX-300 and EAX-400 in solving the 51 small but not easy instances and 27 large instances.

with the special individual (i.e., the special individual in Q-EAX can only be  $p_A$ , not  $p_B$ ). In addition, the result of Q-EAX is the best individual in the population except for the special individual, and Q-EAX will not terminate if the special individual is the optimum solution (if known).

- **Q-EAX+Special:** A variant of Q-EAX that the output is the best individual in the population includes the special individual. Q-EAX+Special terminates when the special individual is the optimum solution. In a word, the difference between Q-EAX+Special and Q-EAX includes: (1) whether the special individual can be the output solution. (2) Whether the algorithm terminates when the special individual is the optimum solution.
- **EAX-LKH:** A variant of RHGA that combines EAX-GA with the LKH algorithm as RHGA does but with no reinforcement learning. The metrics in the EAX-GA and LKH local search are not changed.
- **Alpha-EAX-LKH:** A variant of EAX-LKH using the  $\alpha$ -value (Eq. (1)) to replace the distance metric in EAX-GA.
- **FixQ-EAX-LKH:** A variant of EAX-LKH using the initial Q-value (Eq. (2)) to replace the distance metric in EAX-GA and the  $\alpha$ -value metric in LKH.
- **RHGA- $k$ :** A variant of RHGA that the number of the special individuals is  $k$  (we tested  $k = 3, 5, 20, 50$  in experiments). Each special individual  $x_i$ ,  $i \leq k$ , can be improved by Q-LKH when: (1) it is just initialized, or it was improved by EAX-GA at the last generation. In this case, Q-LKH will try to improve  $x_i$ . (2) It has not been improved for  $M_{gen}$  generations, and the algorithm randomly selects an individual  $x_r$  ( $x_r$  is not a special individual). If  $x_r$  can be improved by Q-LKH and the improved tour is better than  $x_i$ , then replace  $x_i$  with the improved tour. Moreover, when the best individual  $x_{best}$  in the population besides the  $k$  special individuals is better than the best special individual and  $x_{best}$

has not been calculated by Q-LKH, Q-LKH will try to improve  $x_{best}$ . The improved tour will replace  $x_1$  (do not replace  $x_1$  if  $x_{best}$  cannot be improved by Q-LKH).

The termination conditions of all the above algorithms are the same as in RHGA. Since the calculation time of RHGA is a little longer than EAX-300 and close to EAX-400 when solving the tested benchmarks, the calculation times of the variant algorithms, including Alpha-EAX, FixQ-EAX, Q-EAX, Q-EAX+Special, EAX-LKH, Alpha-EAX-LKH, FixQ-EAX-LKH, are also roughly between that of EAX-300 and EAX-400.

We further introduce three hybrid algorithms that combine EAX-GA with LKH in a straightforward manner:

- **Min{EAX, LKH}:** Given an input instance, this algorithm first uses EAX-GA and LKH to calculate the instance 10 times respectively, and then selects the one with the better average solution as the output.
- **LKH+EAX:** A hybrid algorithm that uses LKH to generate the initial population, and then runs EAX-GA starting from this population.
- **EAX+LKH:** A hybrid algorithm that first uses EAX-GA to calculate the input instance until the termination conditions of EAX-GA are reached (or the known optimum is obtained), then runs LKH with its initial solution equal to the best individual. The LKH algorithm in EAX+LKH terminates as EAX-GA does. That is, let  $Gen$  be the number of iterations at which no improvement in the best solution is found by LKH over the recent  $1500/N_{ch}$  iterations. If  $Gen$  has been determined and the best solution does not improve over the last  $G_{max} = Gen/10$  iterations, EAX+LKH terminates.



Table 5

Comparison of RHGA, NeuroLKH\_R, and NeuroLKH\_M. Best results appear in bold.

| Instance | BKS       | RHGA               |                      |        | NeuroLKH_R         |                      |        | NeuroLKH_M         |                      |        |
|----------|-----------|--------------------|----------------------|--------|--------------------|----------------------|--------|--------------------|----------------------|--------|
|          |           | Best (gap%)        | Average (gap%)       | Time   | Best (gap%)        | Average (gap%)       | Time   | Best (gap%)        | Average (gap%)       | Time   |
| u1060    | 224 094   | 224 094 (0.0000)   | 224 094.0 (0.0000)   | 28.4   | 224 094 (0.0000)   | 224 099.1 (0.0023)   | 26.8   | 224 094 (0.0000)   | 224 094.0 (0.0000)   | 21.3   |
| vm1084   | 239 297   | 239 297 (0.0000)   | 239 297.0 (0.0000)   | 19.1   | 239 297 (0.0000)   | 239 379.5 (0.0345)   | 16.5   | 239 297 (0.0000)   | 239 326.4 (0.0123)   | 27.9   |
| pcb1173  | 56 892    | 56 892 (0.0000)    | 56 892.0 (0.0000)    | 21.8   | 56 892 (0.0000)    | 56 892.5 (0.0009)    | 9.2    | 56 892 (0.0000)    | 56 893.0 (0.0018)    | 7.7    |
| d1291    | 50 801    | 50 801 (0.0000)    | 50 801.0 (0.0000)    | 10.6   | 50 801 (0.0000)    | 50 803.4 (0.0047)    | 11.4   | 50 801 (0.0000)    | 50 808.2 (0.0142)    | 6.4    |
| rl1304   | 252 948   | 252 948 (0.0000)   | 252 948.0 (0.0000)   | 9.3    | 252 948 (0.0000)   | 252 953.1 (0.0020)   | 9.2    | 252 948 (0.0000)   | 252 958.2 (0.0040)   | 20.9   |
| rl1323   | 270 199   | 270 199 (0.0000)   | 270 199.0 (0.0000)   | 12.7   | 270 199 (0.0000)   | 270 247.9 (0.0181)   | 16.8   | 270 199 (0.0000)   | 270 204.4 (0.0020)   | 24.2   |
| nrv1379  | 56 638    | 56 638 (0.0000)    | 56 638.0 (0.0000)    | 59.9   | 56 638 (0.0000)    | 56 638.5 (0.0009)    | 20.8   | 56 638 (0.0000)    | 56 638.0 (0.0000)    | 22.9   |
| dca1389  | 5 085     | 5085 (0.0000)      | 5085.0 (0.0000)      | 31.6   | 5087 (0.0393)      | 5087.0 (0.0393)      | 19.6   | 5085 (0.0000)      | 5086.5 (0.0295)      | 11.5   |
| fl1400   | 20 127    | 20 127 (0.0000)    | 20 127.0 (0.0000)    | 74.2   | 20 185 (0.2882)    | 20 185.0 (0.2882)    | 692.5  | 20 189 (0.3080)    | 20 189.0 (0.3080)    | 564.2  |
| dja1436  | 5 257     | 5257 (0.0000)      | 5257.0 (0.0000)      | 25.2   | 5257 (0.0000)      | 5257.2 (0.0038)      | 41.0   | 5257 (0.0000)      | 5257.0 (0.0000)      | 67.6   |
| fra1488  | 4 264     | 4264 (0.0000)      | 4264.0 (0.0000)      | 18.1   | 4264 (0.0000)      | 4264.1 (0.0023)      | 34.1   | 4264 (0.0000)      | 4264.0 (0.0000)      | 2.0    |
| fl1577   | 22 249    | 22 249 (0.0000)    | 22 249.0 (0.0000)    | 67.6   | 22 256 (0.0315)    | 22 256.0 (0.0315)    | 152.8  | 22 698 (2.0181)    | 22 698.0 (2.0181)    | 613.8  |
| rbv1583  | 5 387     | 5387 (0.0000)      | 5387.0 (0.0000)      | 41.8   | 5387 (0.0000)      | 5387.0 (0.0000)      | 36.2   | 5387 (0.0000)      | 5387.1 (0.0019)      | 33.1   |
| fnb1615  | 4 956     | 4956 (0.0000)      | 4956.1 (0.0020)      | 48.4   | 4956 (0.0000)      | 4957.5 (0.0303)      | 114.2  | 4956 (0.0000)      | 4956.0 (0.0000)      | 36.0   |
| rw1621   | 26 051    | 26 051 (0.0000)    | 26 051.0 (0.0000)    | 47.8   | 26 056 (0.0192)    | 26 056.0 (0.0192)    | 735.7  | 26 077 (0.0998)    | 26 077.0 (0.0998)    | 452.9  |
| d1655    | 62 128    | 62 128 (0.0000)    | 62 128.0 (0.0000)    | 47.3   | 62 128 (0.0000)    | 62 128.2 (0.0003)    | 44.0   | 62 128 (0.0000)    | 62 128.0 (0.0000)    | 25.1   |
| vm1748   | 336 556   | 336 556 (0.0000)   | 336 556.0 (0.0000)   | 53.3   | 336 556 (0.0000)   | 336 628.0 (0.0214)   | 42.4   | 336 556 (0.0000)   | 336 556.0 (0.0000)   | 36.7   |
| djc1785  | 6 115     | 6115 (0.0000)      | 6115.0 (0.0000)      | 61.0   | 6115 (0.0000)      | 6115.5 (0.0082)      | 77.9   | 6115 (0.0000)      | 6115.6 (0.0098)      | 42.0   |
| u1817    | 57 201    | 57 201 (0.0000)    | 57 209.1 (0.0142)    | 51.0   | 57 201 (0.0000)    | 57 221.3 (0.0355)    | 159.2  | 57 201 (0.0000)    | 57 239.3 (0.0670)    | 109.8  |
| rl1889   | 316 536   | 316 536 (0.0000)   | 316 536.0 (0.0000)   | 32.7   | 316 638 (0.0322)   | 316 646.8 (0.0350)   | 44.9   | 316 638 (0.0322)   | 316 650.0 (0.0360)   | 58.1   |
| dcc1911  | 6 396     | 6396 (0.0000)      | 6396.0 (0.0000)      | 33.7   | 6396 (0.0000)      | 6396.2 (0.0031)      | 115.0  | 6396 (0.0000)      | 6396.8 (0.0125)      | 22.5   |
| dki1973  | 6 421     | 6421 (0.0000)      | 6421.0 (0.0000)      | 46.2   | 6421 (0.0000)      | 6422.0 (0.0156)      | 241.5  | 6421 (0.0000)      | 6421.0 (0.0000)      | 46.7   |
| mu1979   | 86 891    | 86 891 (0.0000)    | 86 891.0 (0.0000)    | 160.4  | 87 191 (0.3453)    | 87 211.5 (0.3689)    | 412.2  | 87 021 (0.1496)    | 87 021.0 (0.1496)    | 722.7  |
| d2103    | 80 450    | 80 450 (0.0000)    | 80 450.0 (0.0000)    | 48.4   | 80 454 (0.0050)    | 80 454.0 (0.0050)    | 21.7   | 80 459 (0.0112)    | 80 459.0 (0.0112)    | 837.0  |
| u2152    | 64 253    | 64 253 (0.0000)    | 64 253.0 (0.0000)    | 61.4   | 64 253 (0.0000)    | 64 264.4 (0.0177)    | 60.1   | 64 253 (0.0000)    | 64 255.8 (0.0044)    | 244.3  |
| xqc2175  | 6 830     | 6830 (0.0000)      | 6830.0 (0.0000)      | 76.6   | 6830 (0.0000)      | 6830.5 (0.0073)      | 122.5  | 6831 (0.0146)      | 6831.0 (0.0146)      | 23.9   |
| bck2217  | 6 764     | 6764 (0.0000)      | 6764.3 (0.0044)      | 81.9   | 6765 (0.0148)      | 6765.0 (0.0148)      | 35.6   | 6764 (0.0000)      | 6764.3 (0.0044)      | 53.2   |
| xpr2308  | 7 219     | 7219 (0.0000)      | 7219.1 (0.0014)      | 80.4   | 7219 (0.0000)      | 7219.5 (0.0069)      | 47.3   | 7219 (0.0000)      | 7219.9 (0.0125)      | 163.1  |
| ley2323  | 8 352     | 8352 (0.0000)      | 8352.0 (0.0000)      | 49.2   | 8355 (0.0359)      | 8358.4 (0.0766)      | 85.1   | 8355 (0.0359)      | 8355.0 (0.0359)      | 53.7   |
| dea2382  | 8 017     | 8017 (0.0000)      | 8017.0 (0.0000)      | 71.6   | 8018 (0.0125)      | 8019.0 (0.0249)      | 122.4  | 8017 (0.0000)      | 8017.1 (0.0012)      | 117.3  |
| pds2566  | 7 643     | 7643 (0.0000)      | 7643.0 (0.0000)      | 102.6  | 7643 (0.0000)      | 7643.7 (0.0092)      | 135.5  | 7643 (0.0000)      | 7643.3 (0.0039)      | 66.0   |
| mlt2597  | 8 071     | 8071 (0.0000)      | 8071.0 (0.0000)      | 47.6   | 8071 (0.0000)      | 8071.4 (0.0050)      | 142.2  | 8071 (0.0000)      | 8071.0 (0.0000)      | 15.2   |
| bch2762  | 8 234     | 8234 (0.0000)      | 8234.1 (0.0012)      | 131.6  | 8234 (0.0000)      | 8234.0 (0.0000)      | 100.0  | 8234 (0.0000)      | 8234.6 (0.0073)      | 62.2   |
| irw2802  | 8 423     | 8423 (0.0000)      | 8423.0 (0.0000)      | 86.1   | 8423 (0.0000)      | 8424.0 (0.0119)      | 96.8   | 8423 (0.0000)      | 8423.0 (0.0000)      | 114.5  |
| djb2924  | 10 128    | 10 128 (0.0000)    | 10 128.0 (0.0000)    | 127.1  | 10 128 (0.0000)    | 10 128.1 (0.0010)    | 138.3  | 10 128 (0.0000)    | 10 128.7 (0.0069)    | 60.2   |
| xva2993  | 8 492     | 8492 (0.0000)      | 8492.0 (0.0000)      | 128.1  | 8492 (0.0000)      | 8492.0 (0.0000)      | 66.8   | 8492 (0.0000)      | 8492.4 (0.0047)      | 156.0  |
| pcb3038  | 137 694   | 137 694 (0.0000)   | 137 694.0 (0.0000)   | 151.1  | 137 694 (0.0000)   | 137 694.8 (0.0006)   | 217.6  | 137 694 (0.0000)   | 137 694.8 (0.0006)   | 202.7  |
| pia3056  | 8 258     | 8258 (0.0000)      | 8258.5 (0.0061)      | 156.4  | 8261 (0.0363)      | 8261.5 (0.0424)      | 70.2   | 8258 (0.0000)      | 8258.2 (0.0024)      | 164.5  |
| dke3097  | 10 539    | 10 539 (0.0000)    | 10 539.0 (0.0000)    | 127.4  | 10 539 (0.0000)    | 10 539.2 (0.0019)    | 236.6  | 10 539 (0.0000)    | 10 539.0 (0.0000)    | 109.5  |
| lsm3119  | 9 114     | 9114 (0.0000)      | 9114.0 (0.0000)      | 120.3  | 9114 (0.0000)      | 9115.0 (0.0110)      | 249.4  | 9114 (0.0000)      | 9114.1 (0.0011)      | 41.6   |
| lta3140  | 9 517     | 9517 (0.0000)      | 9517.0 (0.0000)      | 134.5  | 9518 (0.0105)      | 9518.0 (0.0105)      | 84.4   | 9517 (0.0000)      | 9517.2 (0.0021)      | 90.0   |
| fdp3256  | 10 008    | 10 008 (0.0000)    | 10 008.1 (0.0010)    | 127.4  | 10 008 (0.0000)    | 10 008.5 (0.0050)    | 572.6  | 10 008 (0.0000)    | 10 011.0 (0.0300)    | 33.6   |
| beg3293  | 9 772     | 9772 (0.0000)      | 9772.2 (0.0020)      | 131.1  | 9773 (0.0102)      | 9774.0 (0.0205)      | 423.4  | 9772 (0.0000)      | 9772.7 (0.0072)      | 121.0  |
| nu3496   | 96 132    | 96 132 (0.0000)    | 96 132.1 (0.0001)    | 169.1  | 96 285 (0.1592)    | 96 285.0 (0.1592)    | 1722.0 | 96 167 (0.0364)    | 96 167.0 (0.0364)    | 1088.5 |
| fjs3649  | 9 272     | 9272 (0.0000)      | 9272.0 (0.0000)      | 176.3  | 9286 (0.1510)      | 9289.0 (0.1833)      | 430.0  | 9274 (0.0216)      | 9278.3 (0.0679)      | 182.6  |
| fjr3672  | 9 601     | 9601 (0.0000)      | 9601.0 (0.0000)      | 158.2  | 9604 (0.0312)      | 9608.0 (0.0729)      | 470.8  | 9602 (0.0104)      | 9602.0 (0.0104)      | 372.4  |
| dlb3694  | 10 959    | 10 959 (0.0000)    | 10 959.3 (0.0027)    | 216.7  | 10 959 (0.0000)    | 10 959.5 (0.0046)    | 228.0  | 10 960 (0.0091)    | 10 960.0 (0.0091)    | 34.0   |
| lhb3729  | 11 821    | 11 821 (0.0000)    | 11 821.0 (0.0000)    | 171.6  | 11 822 (0.0085)    | 11 822.5 (0.0127)    | 511.8  | 11 821 (0.0000)    | 11 821.9 (0.0076)    | 230.5  |
| fl3795   | 28 772    | 28 772 (0.0000)    | 28 777.6 (0.0195)    | 428.3  | 30 623 (6.4333)    | 30 623.0 (6.4333)    | 1981.5 | 29 556 (2.7249)    | 29 556.0 (2.7249)    | 176.4  |
| xqc3891  | 11 995    | 11 995 (0.0000)    | 11 996.1 (0.0092)    | 228.8  | 11 998 (0.0250)    | 11 998.0 (0.0250)    | 82.5   | 11 997 (0.0167)    | 11 997.0 (0.0167)    | 138.6  |
| xua3937  | 11 239    | 11 239 (0.0000)    | 11 239.0 (0.0000)    | 134.8  | 11 239 (0.0000)    | 11 239.0 (0.0000)    | 73.1   | 11 239 (0.0000)    | 11 239.4 (0.0036)    | 213.6  |
| dke3938  | 12 503    | 12 503 (0.0000)    | 12 503.0 (0.0000)    | 187.2  | 12 506 (0.0240)    | 12 506.0 (0.0240)    | 1410.1 | 12 504 (0.0080)    | 12 504.0 (0.0080)    | 123.1  |
| dkt3954  | 12 538    | 12 538 (0.0000)    | 12 538.0 (0.0000)    | 180.3  | 12 538 (0.0000)    | 12 539.2 (0.0096)    | 190.1  | 12 538 (0.0000)    | 12 538.0 (0.0000)    | 106.1  |
| bgb4355  | 12 723    | 12 723 (0.0000)    | 12 723.0 (0.0000)    | 238.1  | 12 725 (0.0157)    | 12 727.5 (0.0354)    | 766.5  | 12 723 (0.0000)    | 12 724.5 (0.0118)    | 253.6  |
| bgd4396  | 13 009    | 13 009 (0.0000)    | 13 009.0 (0.0000)    | 205.0  | 13 011 (0.0154)    | 13 013.7 (0.0361)    | 314.1  | 13 009 (0.0000)    | 13 010.2 (0.0092)    | 182.0  |
| frv4410  | 10 711    | 10 711 (0.0000)    | 10 711.0 (0.0000)    | 224.7  | 10 711 (0.0000)    | 10 713.5 (0.0233)    | 360.3  | 10 711 (0.0000)    | 10 711.4 (0.0037)    | 207.3  |
| bgf4475  | 13 221    | 13 221 (0.0000)    | 13 221.0 (0.0000)    | 222.7  | 13 230 (0.0681)    | 13 230.5 (0.0719)    | 689.3  | 13 221 (0.0000)    | 13 221.8 (0.0061)    | 540.9  |
| ca4663   | 1 290 319 | 1 290 319 (0.0000) | 1 290 319.0 (0.0000) | 395.1  | 1 290 807 (0.0378) | 1 291 931.0 (0.1249) | 557.4  | 1 290 382 (0.0049) | 1 290 597.5 (0.0216) | 398.7  |
| qqd4966  | 15 316    | 15 316 (0.0000)    | 15 316.0 (0.0000)    | 325.7  | 15 344 (0.1828)    | 15 344.0 (0.1828)    | 2406.1 | 15 318 (0.0131)    | 15 318.0 (0.0131)    | 1079.2 |
| fqn5087  | 13 029    | 13 029 (0.0000)    | 13 029.0 (0.0000)    | 428.4  | 13 057 (0.2149)    | 13 057.0 (0.2149)    | 2416.4 | 13 035 (0.0461)    | 13 035.0 (0.0461)    | 877.7  |
| fea5557  | 15 445    | 15 445 (0.0000)    | 15 445.0 (0.0000)    | 293.8  | 15 448 (0.0194)    | 15 450.0 (0.0324)    | 637.3  | 15 445 (0.0000)    | 15 446.0 (0.0065)    | 1280.9 |
| rl5915   | 565 530   | 565 530 (0.0000)   | 565 530.0 (0.0000)   | 289.7  | 566 217 (0.1215)   | 566 608.5 (0.1907)   | 1140.6 | 565 585 (0.0097)   | 565 585.0 (0.0097)   | 692.2  |
| rl5934   | 556 045   | 556 045 (0.0000)   | 556 072.3 (0.0049)   | 476.1  | 556 045 (0.0000)   | 556 058.8 (0.0025)   | 471.8  | 556 045 (0.0000)   | 556 045.9 (0.0002)   | 345.5  |
| tz6117   | 394 718   | 394 718 (0.0000)   | 394 721.2 (0.0008)   | 694.0  | 395 193 (0.1203)   | 395 193.0 (0.1203)   | 2248.3 | 394 720 (0.0005)   | 394 720.0 (0.0005)   | 1185.9 |
| xsc6880  | 21 535    | 21 535 (0.0000)    | 21 535.1 (0.0005)    | 641.7  | 21 544 (0.0418)    | 21 546.0 (0.0511)    | 492.0  | 21 541 (0.0279)    | 21 541.0 (0.0279)    | 730.6  |
| eg7146   | 172 386   | 172 386 (0.0000)   | 172 386.0 (0.0000)   | 1199.0 | 173 594 (0.7008)   | 173 611.0 (0.7106)   | 1636.9 | 173 394 (0.5847)   | 173 394.0 (0.5847)   | 2547.0 |
| bnd7168  | 21 834    | 21 834 (0.0000)    | 21 834.0 (0.0000)    | 396.6  | 21 841 (0.0321)    | 21 842.0 (0.0366)    | 704.1  | 21 838 (0.0183)    | 21 838.0 (0.0183)    | 1774.3 |
| lap7454  | 19 535    | 19 535 (0.0000)    | 19 535.0 (0.0000)    | 457.2  | 19 544 (0.0461)    | 19 545.0 (0.0512)    | 1489.6 | 19 535 (0.0000)    | 19 535.5 (0.0026)    | 591.8  |
| ym7663   | 238 314   | 238 314 (0.0000)   | 238 314.0 (0.0000)   | 1071.9 | 238 811 (0.2085)   | 238 811.0 (0.2085)   | 3623.9 | 238 430 (0.0487)   | 238 430.0 (0.0487)   | 3655.6 |
| pm8079   | 114 855   | 114 855 (0.0000)   | 114 855.3 (0.0003)   | 1301.2 | 115 011 (0.1358)   | 115 011.0 (0.1358)   | 3858.3 | 115 183 (0.2856)   | 115 183.0 (0.2856)   | 3569.4 |
| ida8197  | 22 338    | 22 338 (0.0000)    | 22 338.1 (0.0004)    | 424.8  | 22 348 (0.0448)    | 22 348.0 (0.0448)    | 531.2  | 22 338 (0.0000)    | 22 338.0 (0.0000)    | 211.0  |
| ei8246   | 206 171   | 206 171 (0.0000)   | 206 171.6 (0.0003)   | 1346.3 | 206 179 (0.0039)   | 206 179.0 (0.0039)   | 1844.0 | 206 171 (0.0000)   |                      |        |



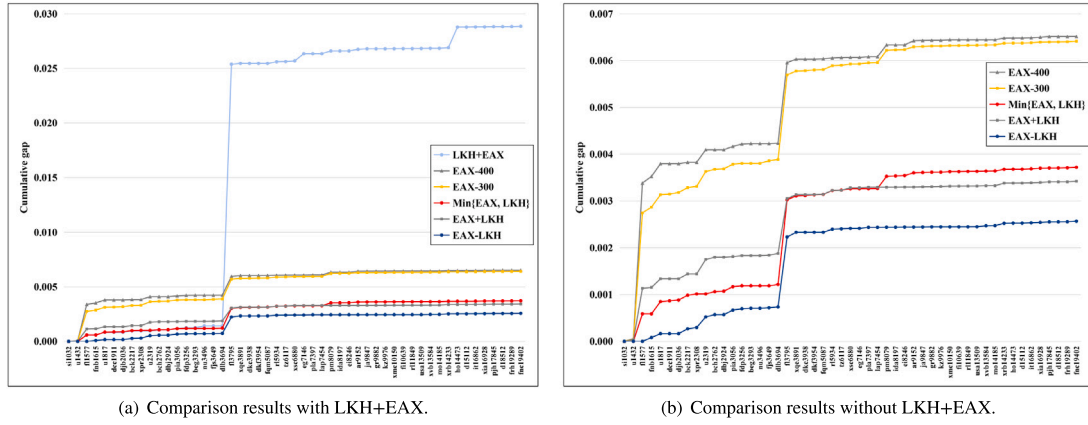


Fig. 5. Analysis on the combination mechanism on 51 *small but not easy* instances.

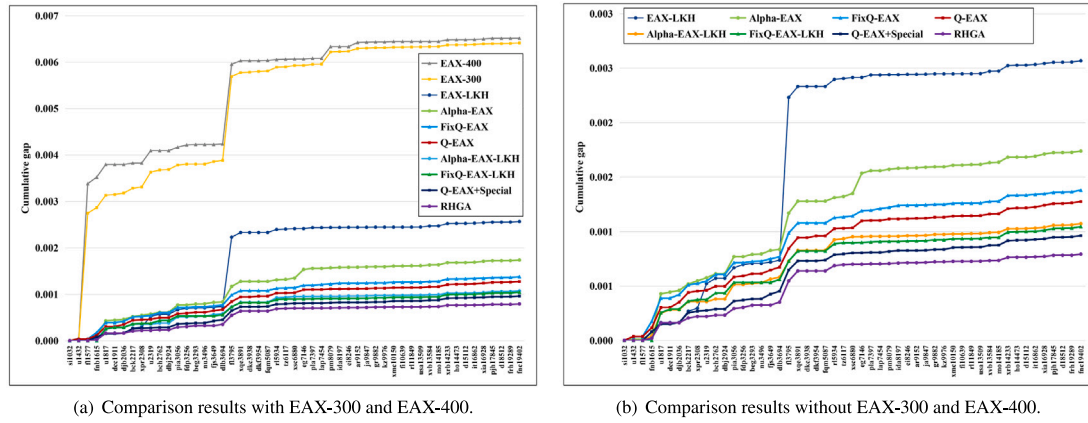


Fig. 6. Analysis on different metrics and the special individual on 51 *small but not easy* instances.

much better performance than  $\text{Min}\{\text{EAX}, \text{LKH}\}$  and  $\text{EAX}+\text{LKH}$ . Note that  $\text{Min}\{\text{EAX}, \text{LKH}\}$  can be regarded as the hybrid solver proposed by Kerschke et al. (2018) consists of EAX-GA and LKH with a perfect prediction model. Therefore, the combination mechanism in RHGA is much more effective than the straightforward hybrid mechanisms we designed for comparison and the hybrid mechanism proposed by Kerschke et al. (2018), and can make better use of the complementarity of EAX-GA and LKH.

### 5.3. Analysis on metrics and special individual

We then compare RHGA with the variant algorithms to evaluate the performance of different metrics in determining the candidate edges, and evaluate the effectiveness of the special individual in RHGA. Fig. 6(a) compares the results of 10 different algorithms, including RHGA, Alpha-EAX, FixQ-EAX, Q-EAX, Q-EAX+Special, EAX-LKH, Alpha-EAX-LKH, FixQ-EAX-LKH, EAX-300, and EAX-400, on the same 51 instances in Fig. 5. We also present the results without EAX-300 and EAX-400 in Fig. 6(b) for a clearer comparison.

The results of EAX-LKH, Alpha-EAX-LKH, FixQ-EAX-LKH, and RHGA in Fig. 6 indicate that, the order with decaying quality of the performance of different metrics is: the adaptive Q-value (updated by reinforcement learning according to Eq. (3)), the initial Q-value (Eq. (2)), the  $\alpha$ -value, and the distance. The results of EAX-300, Alpha-EAX, FixQ-EAX, and Q-EAX can draw the same conclusion. As a result, the EAX-GA algorithm can be improved simply by replacing the distance metric with the  $\alpha$ -value or the initial Q-value, and reinforcement learning can further improve the performance of the initial Q-value. Moreover, the

combination of EAX-GA and LKH by our proposed mechanism is always effective, since EAX-LKH/Alpha-EAX-LKH/FixQ-EAX-LKH/RHGA outperforms EAX-300/Alpha-EAX/FixQ-EAX/Q-EAX significantly.

The results of Q-EAX, Q-EAX+Special, and RHGA in Fig. 6 can further lead to the following comments. First, Q-EAX+Special outperforms Q-EAX, indicating that the special individual can provide better solutions than the other individuals. Second, RHGA outperforms Q-EAX+Special, indicating that the special individual can also improve the population by spreading its genes to other individuals. In summary, the benefits of the special individual are as follows: (1) it can provide the adaptive Q-value for the EAX-GA to improve the performance. (2) it can obtain good solutions since it can be improved by both Q-LKH and EAX-GA (when  $x_1$  is parent  $p_A$ ). (3) it can improve the population by providing its high-quality genes for other individuals (when  $x_1$  is parent  $p_B$ ).

### 5.4. Comparison with multiple special individuals

We then compare RHGA with the RHGA- $k$  algorithms, including RHGA-3, RHGA-5, RHGA-20, and RHGA-50, on the same 51 instances in Fig. 5, to analyze the influence of the number of special individuals on the performance. The comparison results are shown in Fig. 7.

From the results we can observe that:

(1) RHGA and RHGA-3 show similar performance in solving the 51 instances. The algorithms with a larger number of special individuals show better performance in solving the instances with relatively small scales. For example, the RHGA-5 is good at solving the instances with less than 3000 cities, and the RHGA-20 and RHGA-50 are good at solving the instances with less than 6000 cities. However, neither of the

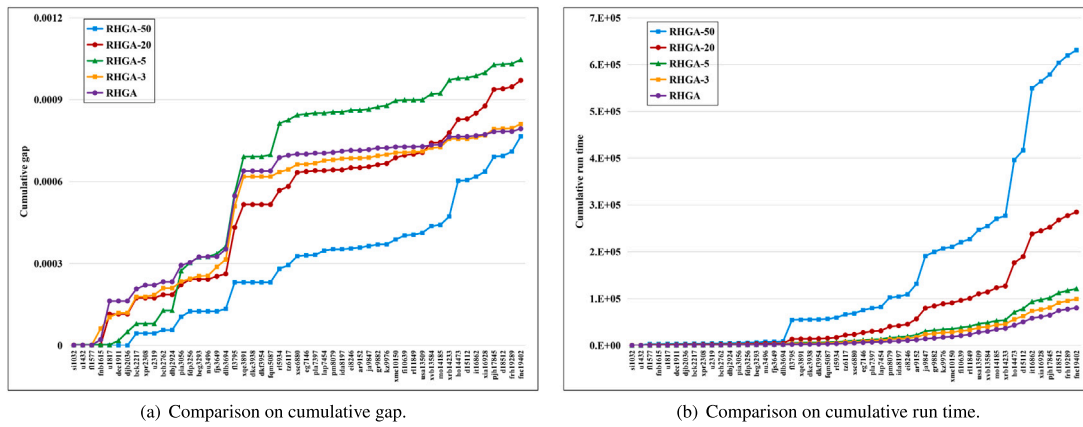


Fig. 7. Comparison results of RHGA and RHGA-3/5/20/50 in solving the 51 *small but not easy* instances.

three algorithms with a larger number of special individuals is good at solving the instances with more than 14,000 cities, indicating that for solving relatively large and complex instances, an excessive proportion of special individuals in the population may reduce the algorithm's performance.

(2) The calculation time of the RHGA- $k$  algorithm increases rapidly as  $k$  increases. Specifically, the calculation time of RHGA-3/5/20/50 is about 24/51/256/687% more than that of RHGA.

In summary, setting multiple special individuals in the population is not reasonable and time-consuming. Thus it is suitable and effective to set only one special individual in the population, which can help the algorithm obtain a good performance without reducing the efficiency.

## 6. Conclusion

In this paper, we address the famous NP-hard traveling salesman problem and propose a Reinforced Hybrid Genetic Algorithm (RHGA) that combines reinforcement learning with two state-of-the-art heuristic algorithms, namely EAX Genetic Algorithm (EAX-GA) and LKH local search heuristic, in a more interactive manner. The EAX-GA and LKH are integrated with the help of a special and unique individual, which can be improved by both the genetic algorithm and the local search algorithm. In our proposed hybrid mechanism, the population provides diverse and high-quality initial solutions for the LKH local search algorithm, and the local search algorithm leads the population to converge to better results. In a word, the two state-of-the-art TSP heuristics, EAX-GA and LKH, can boost each other in the proposed hybrid mechanism. Moreover, the Q-learning algorithm is applied to learn an adaptive Q-value to replace the distance metric used in the process of merging sub-tours in EAX-GA and the  $\alpha$ -value used in LKH for determining the candidate cities. As a result, our reinforcement learning method can improve both the EAX-GA and LKH algorithms by providing better candidate edges.

Extensive empirical results demonstrate the superiority of RHGA over the powerful EAX-GA and LKH algorithms, as well as a state-of-the-art (deep) learning based algorithm, NeuroLKH, for solving the TSP. Further and extensive ablation studies are adopted to show the effectiveness of the proposed hybrid mechanism and the reinforcement learning method, demonstrating that the proposed hybrid mechanism can make full use of the complementary of EAX-GA and LKH, and the setting of only one special individual is reasonable and efficient.

In future work, the proposed mechanism of combining genetic algorithms with local search could be applied to solve various combinatorial optimization problems, and the method of combining reinforcement learning with the key process of heuristics could also be applied to improve other heuristic algorithms.

## CRedit authorship contribution statement

**Jiongzi Zheng:** Conceptualization of this study, Methodology, Software, Writing and revision. **Jialun Zhong:** Writing and revision. **Menglei Chen:** Software. **Kun He:** Conceptualization of this study, Methodology, Supervision, Writing and revision.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request

## Acknowledgments

This work is supported by National Natural Science Foundation (U22B2017) and MSRA Collaborative Research 2022 (100338928).

## References

- Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S., 2017. Neural combinatorial optimization with reinforcement learning. In: Fifth International Conference on Learning Representations, Workshop Track Proceedings, ICLR (Workshop) 2017.
- Bengio, Y., Lodi, A., Prouvost, A., 2021. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European J. Oper. Res.* 290 (2), 405–421.
- de O da Costa, P.R., Rhuggenaath, J., Zhang, Y., Akcay, A., 2020. Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning. In: The Twelfth Asian Conference on Machine Learning, ACML 2020, Vol. 129. pp. 465–480.
- Freisleben, B., Merz, P., 1996. New genetic local search operators for the traveling salesman problem. In: The Fourth International Conference on Parallel Problem Solving from Nature, PPSN 1996, Vol. 1141. pp. 890–899.
- Gama, R., Fernandes, H.L., 2021. A reinforcement learning approach to the orienteering problem with time windows. *Comput. Oper. Res.* 133, 105357.
- He, P., Hao, J., 2022. Hybrid search with neighborhood reduction for the multiple traveling salesman problem. *Comput. Oper. Res.* 142, 105726.
- Held, M., Karp, R.M., 1970. The traveling-salesman problem and minimum spanning trees. *Oper. Res.* 18 (6), 1138–1162.
- Helsgaun, K., 2000. An effective implementation of the lin-kernighan traveling salesman heuristic. *European J. Oper. Res.* 126 (1), 106–130.
- Helsgaun, K., 2009. General k-opt submoves for the lin-kernighan TSP heuristic. *Math. Program. Comput.* 1 (2–3), 119–163.
- Helsgaun, K., 2017. An Extension of the Lin-Kernighan-Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems. Technical Report, Roskilde University.
- Ilin, V., Simic, D., Simic, S.D., Simic, S., 2020. Hybrid genetic algorithms and tour construction and improvement algorithms used for optimizing the traveling salesman problem. In: Fifteenth International Conference on Soft Computing Models in Industrial and Environmental Applications, SOCO 2020, Vol. 1268. pp. 530–539.

- Kerschke, P., Kotthoff, L., Bossek, J., Hoos, H.H., Trautmann, H., 2018. Leveraging TSP solver complementarity through machine learning. *Evol. Comput.* 26 (4).
- Khalil, E.B., Dai, H., Zhang, Y., Dilkina, B., Song, L., 2017. Learning combinatorial optimization algorithms over graphs. In: *Advances in Neural Information Processing Systems* 30, NIPS 2017. pp. 6348–6358.
- Lin, S., 1965. Computer solutions of the traveling salesman problem. *Bell Syst. Tech. J.* 44 (10), 2245–2269.
- Lin, S., Kernighan, B.W., 1973. An effective heuristic algorithm for the traveling-salesman problem. *Oper. Res.* 21 (2), 498–516.
- Liu, F., Zeng, G., 2009. Study of genetic algorithm with reinforcement learning to solve the TSP. *Expert Syst. Appl.* 36 (3), 6995–7001.
- Maekawa, K., Mori, N., Tamaki, H., Kita, H., Nishikawa, Y., 1996. A genetic solution for the traveling salesman problem by means of a thermodynamical selection rule. In: *IEEE International Conference on Evolutionary Computation, ICEC 1996*. pp. 529–534.
- Mazyavkina, N., Sviridov, S., Ivanov, S., Burnaev, E., 2021. Reinforcement learning for combinatorial optimization: A survey. *Comput. Oper. Res.* 134, 105400.
- Merz, P., Freisleben, B., 1997. Genetic local search for the TSP: new results. In: *IEEE International Conference on Evolutionary Computation, ICEC 1997*. pp. 159–164.
- Nagata, Y., 2006. New EAX crossover for large TSP instances. In: *The Ninth International Conference on Parallel Problem Solving from Nature, PPSN 2006*, Vol. 4193. pp. 372–381.
- Nagata, Y., Bräysy, O., Dullaert, W., 2010. A penalty-based edge assembly memetic algorithm for the vehicle routing problem with time windows. *Comput. Oper. Res.* 37 (4), 724–737.
- Nagata, Y., Kobayashi, S., 1997. Edge assembly crossover: A high-power genetic algorithm for the travelling salesman problem. In: *The Seventh International Conference on Genetic Algorithms, ICGA 1997*. pp. 450–457.
- Nagata, Y., Kobayashi, S., 2013. A powerful genetic algorithm using edge assembly crossover for the traveling salesman problem. *INFORMS J. Comput.* 25 (2), 346–363.
- Nguyen, H.D., Yoshihara, I., Yamamori, K., Yasunaga, M., 2007. Implementation of an effective hybrid GA for large-scale traveling salesman problems. *IEEE Trans. Syst. Man Cybern. B* 37 (1), 92–99.
- Rashid, M.H., Mosteiro, M.A., 2017. A greedy-genetic local-search heuristic for the traveling salesman problem. In: *2017 IEEE International Symposium on Parallel and Distributed Processing with Applications and 2017 IEEE International Conference on Ubiquitous Computing and Communications, ISPA/IUCC 2017*. pp. 868–872.
- Sui, J., Ding, S., Liu, R., Xu, L., Bu, D., 2021. Learning 3-opt heuristics for traveling salesman problem via deep reinforcement learning. In: *The Thirteenth Asian Conference on Machine Learning, ACML 2021*, Vol. 157. pp. 1301–1316.
- Sutton, R.S., Barto, A.G., 1998. *Reinforcement Learning - an Introduction*. In: *Adaptive Computation and Machine Learning*. MIT Press.
- Tsai, H., Yang, J., Tsai, Y., Kao, C., 2004. An evolutionary algorithm for large traveling salesman problems. *IEEE Trans. Syst. Man Cybern. B* 34 (4), 1718–1729.
- Ulder, N.L., Aarts, E.H., Bandelt, H.-J., Laarhoven, P.J.V., Pesch, E., 1990. Genetic local search algorithms for the traveling salesman problem. In: *International Conference on Parallel Problem Solving from Nature, PPSN 1990*, Vol. 496. pp. 109–116.
- Vinyals, O., Fortunato, M., Jaitly, N., 2015. Pointer networks. In: *Advances in Neural Information Processing Systems* 28, NIPS 2015. pp. 2692–2700.
- Wang, Y., 2014. The hybrid genetic algorithm with two local optimization strategies for traveling salesman problem. *Comput. Ind. Eng.* 70, 124–133.
- Whitley, L.D., Hains, D., Howe, A.E., 2010. A hybrid genetic algorithm for the traveling salesman problem using generalized partition crossover. In: *The Eleventh International Conference on Parallel Problem Solving from Nature, Part I, PPSN I 2010*, Vol. 6238. pp. 566–575.
- Xin, L., Song, W., Cao, Z., Zhang, J., 2021a. Multi-decoder attention model with embedding glimpse for solving vehicle routing problems. In: *The Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021*. pp. 12042–12049.
- Xin, L., Song, W., Cao, Z., Zhang, J., 2021b. NeuroLKH: Combining deep learning model with lin-kernighan-helsgaun heuristic for solving the traveling salesman problem. In: *Advances in Neural Information Processing Systems* 34, *NeurIPS 2021*. pp. 7472–7483.
- Zhao, J., Mao, M., Zhao, X., Zou, J., 2021. A hybrid of deep reinforcement learning and local search for the vehicle routing problems. *IEEE Trans. Intell. Transp. Syst.* 22 (11), 7208–7218.
- Zheng, J., He, K., Zhou, J., Jin, Y., Li, C., 2021. Combining reinforcement learning with lin-kernighan-helsgaun algorithm for the traveling salesman problem. In: *The Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021*. pp. 12445–12452.